Towards Efficient Performance Debugging: A Dynamic and Adaptive Execution Tracing Technique with Large-Scale Comparative Validation

Mohammed Adib Khan[1†], Morteza Noferesti[1*†], Naser Ezzati-Jivan[1†]

[1*]Computer Science Department, Brock University, 1812 Sir Isaac Brock Way, St. Catharines, L2S-3A1, ON, Canada.

*Corresponding author(s). E-mail(s): mnoferesti@brocku.ca; Contributing authors: khanm@brocku.ca; nezzatijivan@brocku.ca; [†]These authors contributed equally to this work.

Abstract

Performance debugging in production systems is hampered by high tracing overhead and the challenge of determining *when* to collect data. This paper presents an extended evaluation of the dynamic adaptive execution tracing (DAET) framework. The framework operates in three phases: (1) Tracing-Function Selection via call-stack profiling and coefficient-of-variation (CoV) ranking, (2) Change Detection based on anomaly scoring with a dynamic threshold, and (3) Trace Configuration Adjustment that activates tracepoints only when the anomaly score exceeds a tolerance-based gate.

This journal extension provides a large-scale, statistically grounded evaluation of multiple change-detection engines (ARIMA, LAST, SMA, EWMA, and a compact Transformer) across two Mozilla performance-telemetry benchmarks under a unified 60%/20%/20% chronological protocol, including dual-dataset validation, replay-based deployment analysis, and a fair multi-seed Transformer comparison. On Firefox-Android (136 of 342 test-alerted signatures), SMA($\boldsymbol{w} = \mathbf{40}$) achieves the highest detection rate (72.8%) among classical methods, with EWMA($\boldsymbol{\alpha} = \mathbf{0.05}$) performing comparably, while ARIMA outperforms LAST. The compact Transformer (WINDOW=20, $\boldsymbol{d}_{\text{model}} = \mathbf{16}$) achieves a detection rate of 74.4%, competitive with the best classical methods and stable across five seeds. On mozilla-beta, SMA again leads (91.7%), followed closely by EWMA and ARIMA, while LAST remains weakest.

The qualitative ranking SMA $\approx$ EWMA $\geq$ ARIMA $>$ LAST is consistent across both datasets, providing preliminary evidence of robustness within the Mozilla CI context; all metrics measure *agreement with Treeherder alerts* rather than ground-truth anomaly validity, and precision remains below 0.12 across all methods.
All detectors reduce trace storage by 67–90%, with classical methods achieving higher reduction than the Transformer (67–71%). The ablation study shows that default parameters correspond to a high-recall operating point, while alternative settings trade recall for more selective detection.
These results indicate that simple statistical methods provide computationally efficient baselines for adaptive tracing that are competitive with lightweight learned models under fair evaluation; their utility for actual debugging effectiveness requires further validation beyond alert-agreement metrics.

Keywords: kernel tracing, application tracing, execution tracing, adaptive performance monitoring, performance debugging, anomaly detection, ARIMA, change detection, Mozilla telemetry

1 Introduction

Modern software systems rely on continuous performance monitoring to detect regressions before they impact end users. Execution tracing, recording timestamped function invocations, system calls, and hardware events, provides the most detailed diagnostic data for root-cause analysis [1–3]. However, always-on tracing imposes non-trivial runtime overhead: the LTTng kernel tracer requires as little as 158 ns per tracepoint materialization [4], yet at high event rates the cumulative I/O and storage cost quickly becomes prohibitive [5]. The fundamental dilemma is that tracing must be enabled before a regression manifests to capture causally relevant data, yet leaving it enabled at all times wastes resources and generates enormous volumes of irrelevant data.

1.1 Foundation

In our prior work [6] we introduced the Dynamic Adaptive Execution Tracing (DAET) framework, which addresses this dilemma by automating the decision of *which functions* to monitor and *when* to activate their tracepoints. The framework consists of three phases: a profiling phase that builds a ranked candidate list of functions via call-stack sampling and CoV-based prioritisation; a change-detection phase that feeds function-execution time-series data into a pluggable change-detection engine (ARIMA(1,1,1) in the original work) and maintains a decaying anomaly score; and a trace-configuration phase that toggles tracepoints when the anomaly score crosses a predefined threshold. Four Firefox/Bugzilla case studies demonstrated that the framework (a) correctly ranks known culprit functions in the top-5% monitored set, (b) triggers tracing only during anomalous intervals (44% storage reduction on a representative tracepoint), and (c) handles both application-level and kernel-level components through a multi-level mechanism. Those case studies were illustrative rather

than prospective: a known bug report was used *post hoc* to confirm that the framework would have surfaced the culprit. Rigorous, prospective, large-scale evaluation was explicitly deferred as future work [6].

1.2 Motivation and Scope

This journal extension is motivated by three unresolved questions left open by the conference version of the DAET framework, each addressing a key gap in evaluation, deployment, and robustness.

Engine choice. The conference paper adopts ARIMA as the change-detection engine due to its ability to model temporal trends while remaining sensitive to regime shifts. However, simpler alternatives, such as last-value (LAST), simple moving average (SMA), and exponentially weighted moving average (EWMA), exhibit similar predictive behaviour at substantially lower computational cost. It remains unclear whether the added complexity of ARIMA translates into measurably superior detection performance on large-scale, real-world telemetry.

Deployment trade-offs. Accurate detection alone is insufficient for practical performance debugging: the benefit depends on whether tracing is *active at the right time*. Prior work has not systematically quantified the trade-off between trace storage reduction and alert coverage induced by different detectors under a unified replay setting, leaving the deployment implications of detector choice largely unexplored.

Parameter robustness. Key parameters of the anomaly-scoring mechanism, including the decay factor β, detection threshold θ, and tolerance window τ, were selected empirically in the conference study. A systematic analysis of their sensitivity and the stability of the operating region across a large dataset is lacking, limiting confidence in the robustness of the proposed configuration.

Together, these questions motivate a comprehensive empirical evaluation of change-detection methods within the DAET framework, focusing on accuracy, deployment efficiency, and parameter stability.

1.3 Research Questions

This journal paper addresses the following extension research questions:

RQ1 Among ARIMA, LAST, SMA, and EWMA, which statistical change detector achieves the best precision, recall, F1 score, and false-alarm rate when applied to 214 Mozilla Firefox-Android regression-performance signatures? A compact Transformer encoder is additionally evaluated under a fair five-seed 60%/20%/20% chronological protocol across both datasets (Section 5.6).
RQ2 Are the performance differences between detectors statistically significant at the $\alpha = 0.05$ level?
RQ3 How sensitive are the results to the key algorithmic parameters (β, θ, τ, ARIMA order, SMA window)?
RQ4 When each detector's binary output is used to gate trace recording, what storage-reduction versus alert-coverage trade-off does it produce under a replay simulation on the Mozilla dataset?

1.4 Contributions

The primary contribution of this journal extension over [6] is an *empirical comparative extension*: a large-scale validation of the DAET framework's detection phase, a comparative detector study, a formal statistical analysis, and a replay-based storage/coverage analysis.

Specifically, the extension contributes:

1. A reproducible large-scale comparative validation of four classical change detectors, ARIMA(1,1,1), LAST, SMA, EWMA, on 214 Mozilla Firefox-Android Treeherder telemetry signatures (214 time-series $\times$ 4 methods = 856 evaluated detector–signature pairs).
2. A formal statistical analysis: 2,000-resample bootstrap 95% confidence intervals for precision, recall, F1, and false-alarm rate; Wilcoxon signed-rank tests for all six pairwise comparisons among the four statistical detectors, confirming that ARIMA vs. SMA differences in F1 and precision are *not* statistically significant ($p > 0.05$).
3. An ablation study sweeping $\beta \in \{0.1, 0.2, 0.3, 0.5, 0.7\}$, $\theta \in \{0.10, \ldots, 0.50\}$, $\tau \in \{1, 2, 3, 5, 7, 10\}$, five ARIMA orders, and six SMA windows, characterising the stable operating regions of the parameter space and the operational tradeoffs involved in parameter selection.
4. A trace-on replay experiment on the full test-set chronology for the four statistical detectors, measuring storage reduction (89-91%), trigger frequency ($\approx$5.7-6.1%), and alert coverage (70-93%), plus an efficiency-frontier analysis characterising the storage–coverage Pareto trade-off.
5. A three-way chronological split validation (60%/20%/20%) in which SMA w and EWMA α are independently tuned on a held-out validation set and metrics are reported on a clean held-out test set. This directly addresses the internal validity limitation of exploratory parameter selection, and reveals that properly tuned EWMA($\alpha = 0.05$) achieves F1 comparable to SMA($w = 40$) on Firefox-Android, and likewise on mozilla-beta, partially revising the original cross-detector ranking.
6. A *fair multi-seed* (five seeds) dual-dataset three-way evaluation of a compact Transformer encoder under the *identical* 60%/20%/20% chronological split protocol as all classical methods, closing the fairness gap of the prior single-seed 70%/30% Transformer evaluation. The compact Transformer uses the same Algorithm-3 anomaly-score gating as the classical detectors, with architecture and hyperparameters tuned per dataset on the held-out validation set. On Firefox-Android (WINDOW=20, $d_{\text{model}} = 16$) the compact Transformer achieves F1 = 0.105 ($\pm$0.002) and detection rate 74.4%; on mozilla-beta (WINDOW=10, $d_{\text{model}} = 32$) F1 = 0.079 ($\pm$0.001). The qualitative ranking SMA $\approx$ EWMA $\geq$ ARIMA $>$ LAST is consistent across both datasets.

1.5 Paper Outline

Section 2 reviews related work. Section 3 describes the DAET framework. Section 4 summarises the four case studies from the conference paper. Section 5 presents the large-scale comparative evaluation (RQ1–RQ2). Section 6 reports ablation results

(RQ3). Section 7 describes the replay experiment (RQ4). Section 8 discusses findings. Section 9 addresses threats to validity. Section 10 concludes. The three-way evaluation including the fair compact-Transformer multi-seed experiment is reported inline within the large-scale evaluation (Section 5.6).

2 Related Works

2.1 Software Analysis

Previous work on software analysis includes static and dynamic analysis approaches. Static analysis tools such as Facebook Infer [7] and FindBugs [8] identify performance issues by detecting anti-patterns in source code, such as inefficient loops or wasteful call sequences. While they impose no runtime impact, they often generate false positives due to lack of runtime context [9–11].

Mace et al. [12] proposed pivot tracing, which instruments systems without kernel modifications using an SQL-like query language. However, it requires prior knowledge of the program's source code. Mirgorodskiy et al. [13] introduced self-propelled instrumentation, where an autonomous agent propagates through code via execution flow; this fails when all execution flows halt simultaneously [9].

Dynamic runtime analysis [5, 14] monitors program behaviour to pinpoint root causes. DTrace [1] dynamically modifies the OS kernel to place probes only when needed. LTTng [2] and Ftrace [3] provide low-overhead kernel-level tracing. While dynamic approaches can address issues in production environments, they introduce runtime overhead [5, 14].

2.2 Clustering, Anomaly, and Trend Detection

Analysing system-level metrics provides insights for predicting faults and detecting performance regressions. Nguyen et al. [15] used low-level metrics to detect abnormal change propagation paths. Zhang et al. [16] applied K-Means density clustering to detect anomalies in performance data. Tate et al. [17] employed a cohort intelligence model fed with kernel trace data.

Ehlers et al. [18] used ARIMA to detect trends in response times and adaptively monitor system performance—an approach that inspires our own anomaly-score function. Their work focuses on application-level tracing only; our framework extends to multi-level (application *and* kernel) adaptive tracing.

Ates et al. [19] proposed Pythia, a real-time workflow-centric tracing framework that continuously gathers critical-path skeletons and groups them by performance expectation. Pythia is limited to application-level tracing and assumes no problematic groups in the first run, whereas DAET is fully online and does not require prior labelling.

2.3 Time-Series Anomaly Detection in Software Systems

Anomaly detection in time-series metrics from production systems has attracted considerable attention. Liu et al. [20] introduced Opprentice, a supervised offline learning

approach for selecting operator-preferred anomaly detectors from a library of candidates applied to performance metrics. Lavin and Ahmad [21] proposed NAB, a benchmark for evaluating real-time anomaly detectors on streaming data, providing a standardised scoring framework that accounts for detection latency. KDD datasets and the Yahoo EGADS system [22] demonstrate the breadth of statistical and ML-based approaches applied to large-scale time-series anomaly detection in production. Cid-Fuentes et al. [23] proposed adaptive SVM-based anomaly detection in distributed systems, the closest prior work to our setting, though their approach requires labelled training data that is unavailable in our Mozilla deployment scenario. DAET's change-detection phase differs in being fully unsupervised and online, requiring no labelled anomalies, no manual threshold selection beyond the ablation-justified defaults, and no periodic retraining.

2.4 Metrics Selection

Selecting the right metrics is critical for informed adaptive tracing decisions. Laguna et al. [24] used PCA on abnormal/normal traces to reduce dimensionality; however, their approach requires significant manual effort. Munawar et al. [25] evaluated random, correlation-based, hierarchical, and MST-based selection methods, finding MST-based pairwise correlation selection superior, inspiring our own correlation-based filtering. Dymshits et al. [26] observed that system call counts become stable unless perturbed by external factors, supporting our assumption of metric stability in healthy systems. Han et al. [27] proposed StackMine, which mines call-stack snapshots to estimate function execution times, an idea we adapt in Phase 1 of our framework.

3 Adaptive Execution Tracing Framework

This section describes the three-phase DAET framework. Figure 1 shows the overall architecture.

3.1 Phase 1: Tracing-Function Selection

3.1.1 Call-Stack Monitoring

A suitable snapshot period is determined by the application's characteristics (e.g., 10 ms for a browser). The application is executed under representative workloads, and all observed function and system calls are recorded.

3.1.2 Family-Group Identification and Sorting

A *family group* captures the parent–child call hierarchy. For a snapshot $SP = \{\textit{Call-Stack}^i \mid 1 \leq i \leq n\}$, the family group of $\textit{Call-Stack}^1 = \langle F^a, F^b, F^c \rangle$ is $\{(F^a, F^b), (F^b, F^c)\}$. Groups are sorted by invocation frequency, prioritising functions with the highest potential performance impact. Algorithm 1 formalises this procedure.

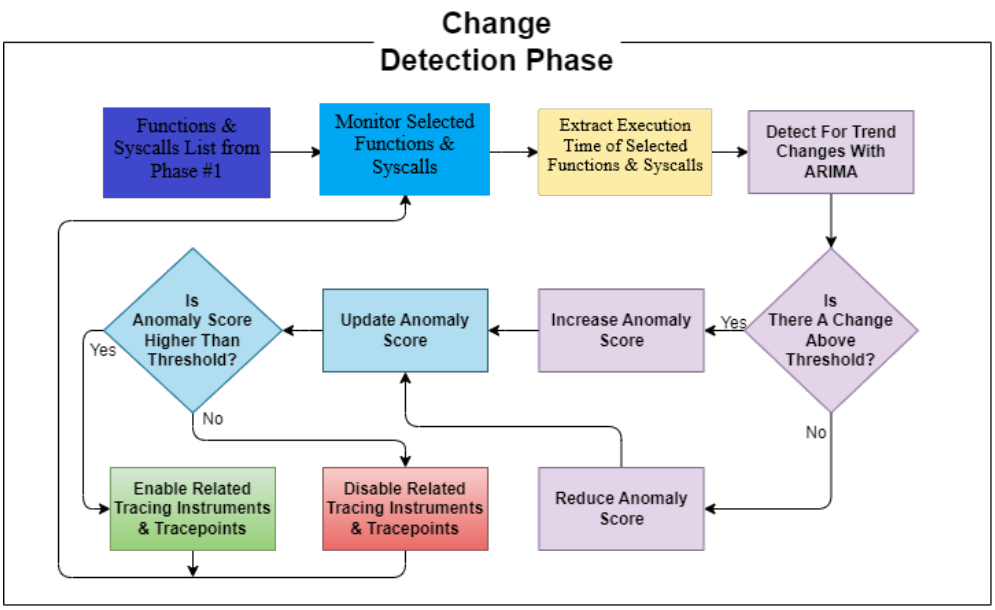


Fig. 1: Change detection phase overview: a pluggable change-detection engine (ARIMA(1,1,1) shown) drives the anomaly score, which gates the trace configuration.

Algorithm 1 Family-Group Identification & Sorting

```
FG_List ← [];    FG_Count ← []
for all Snapshot ∈ CallStackSnapshots do
    for all FG ∈ ExtractFamilyGroups(Snapshot) do
        if FG ∉ FG_List then append FG to FG_List; set count to 1
        else increment FG_Count[FG]
Sort FG_List descending by FG_Count
return FG_List
```

3.1.3 CoV Ranking

Functions are further ranked by their Coefficient of Variation (CoV) of execution time $CoV_M = \sigma_M/\mu_M$, ensuring the most volatile functions are monitored (Algorithm 2).

3.1.4 List Cutoff

A cutoff (e.g., top-5% of all monitored functions) ensures that only the most performance-critical candidates are tracked, limiting monitoring overhead without sacrificing coverage of high-impact calls.

Algorithm 2 CoV Ranking of Function Calls

```
for all f ∈ FunctionExecTimeList do
    μ_f ← mean execution time of f
    σ_f ← standard deviation of f
    CoV_f ← σ_f / μ_f
Sort FunctionExecTimeList descending by CoV
return top-k% of list (e.g., k = 5)
```

3.2 Phase 2: Change Detection

3.2.1 ARIMA(1,1,1) Model

For each monitored function, an ARIMA(1,1,1) model predicts the next execution-time sample based on historical data:

$$\hat{x}_{t+1} = x_t + \varphi\,\Delta x_t + \theta\,\varepsilon_t, \qquad \varepsilon_t = x_t - \hat{x}_t. \tag{1}$$

First-order differencing captures linear trends; the model flags deviations from the expected trend without adapting to them, preserving sensitivity to genuine regime shifts. A dynamic threshold adaptively scales with recent variability:

$$\Theta_t = \max\big(\Theta_{\min},\ k_\sigma \cdot \mathrm{std}(\delta_{t-w:t})\big), \tag{2}$$

where $\delta_t = |x_t - \hat{x}_t|$, $w = 30$ (lookback window), and $k_\sigma = 3.0$. A reading is flagged anomalous when $\delta_t > \Theta_t$.

Why ARIMA? ARIMA predicts based on past trends and—unlike machine-learning approaches—does not adapt to new changes, which preserves detection sensitivity. ARIMA identifies *all* trend changes regardless of cause (load shift, anomaly, attack), signalling that additional tracing data are needed. We select ARIMA(1,1,1) rather than higher-order variants so that gradual, organic workload growth is accommodated (the integrated component re-centres predictions) while quadratic anomalies remain detectable.

3.2.2 Anomaly Score

The anomaly score is an exponentially weighted accumulator that rises when anomalies occur frequently and decays otherwise:

$$s_t = \beta \cdot \mathbf{1}[\delta_t > \Theta_t] + (1-\beta) \cdot s_{t-1}, \tag{3}$$

where $\beta \in (0,1)$ controls the reaction rate. Tracing is activated when $s_t \geq \theta$ and deactivated when the score falls below θ. This gating prevents over-reaction to single spikes while ensuring persistent anomalies are always captured (see Figure 2).

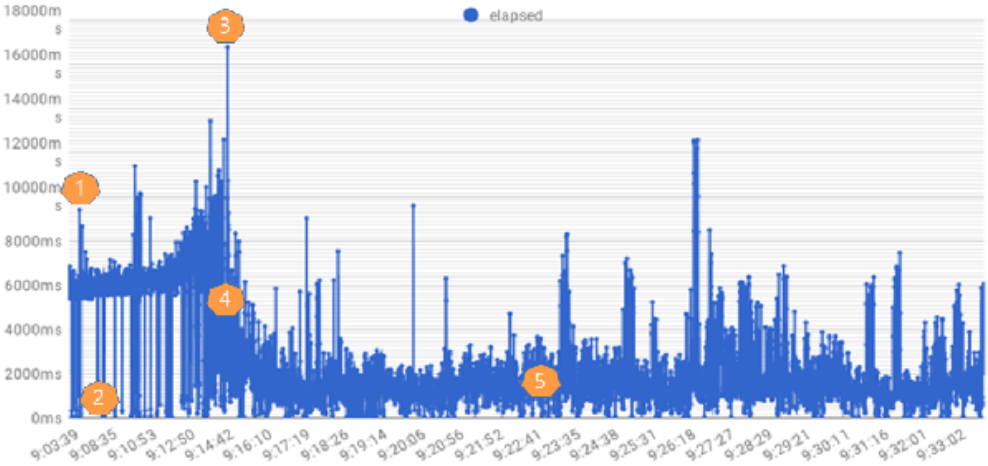


Fig. 2: Illustrative response-time series with five labelled events. A single spike (1) is insufficient to activate tracing; repeated anomalies (3) raise the score above threshold; the score must fall below threshold (5) before tracing is deactivated.

Algorithm 3 Online Change Detection and Trace Gating

Initialise $s_t \leftarrow 0$ for all monitored functions
for all incoming sample x_t per function **do**
  $\hat{x}_t \leftarrow \mathrm{ARIMA}(x_{t-1}, x_{t-2}, \ldots)$
  $\delta_t \leftarrow |x_t - \hat{x}_t|$
  $\Theta_t \leftarrow \max(\Theta_{\min},\ k_\sigma \cdot \mathrm{std}(\delta_{t-w:t}))$
  $s_t \leftarrow \beta \cdot \mathbf{1}[\delta_t > \Theta_t] + (1-\beta) \cdot s_{t-1}$
  if $s_t \geq \theta$ **then**
    Enable tracepoint(s) for this function's family-group
  else
    Disable tracepoint(s)

3.3 Phase 3: Trace Configuration Change

When the change-detection phase identifies a trend deviation, tracing is activated for the associated function family-groups up to a pre-set depth limit. At the system level, detected system-call anomalies are mapped to the corresponding kernel component via the Linux system-call table [28], enabling targeted kernel-level tracing. Algorithm 3 combines all three decisions into a single online loop.

4 Prior Qualitative Evaluation

The conference paper [6] validated the framework through four case studies on Firefox and a controlled Apache server. These were *illustrative demonstrations*: in each case, a known Bugzilla performance-bug report was used post hoc to confirm that the

Case	Bug / Scenario	Detected?	Candidate-list overhead	ARIMA storage reduction
1	Apache CPU/disk/network trend changes	Yes (21/35 metrics)	66% (nmon metric selection)	—
2	Firefox VSync driver slowdown (PDF load)	Yes (rank 14 of 8,231)	95% (CoV cutoff, 410 monitored)	44%
3	Firefox 3D-rendering slowdown	Yes (rank 6)	95% (CoV cutoff)	—
4	Firefox startup slowdown (DNS failure)	Yes (`__sys_sendmsg` → `net/socket.c`)	95% (CoV cutoff)	Multi-level detection

Table 1: Summary of four qualitative case studies from [6]. All overhead reductions are relative to always-on monitoring of the full candidate set.

framework ranked the responsible function highly and triggered tracing at the correct time windows. Table 1 summarises the key results; the subsections below restore the concrete metric-level findings that are most informative for practical deployment.

These case studies confirm that the framework correctly narrows a large candidate space to a tractable monitored set, that the anomaly-score mechanism triggers and releases tracing at appropriate times, and that multi-level (application + kernel) detection is feasible. Rigorous prospective evaluation at scale is what motivates the remainder of this paper.

4.1 Case Study 1: System-Level Overhead Characterisation

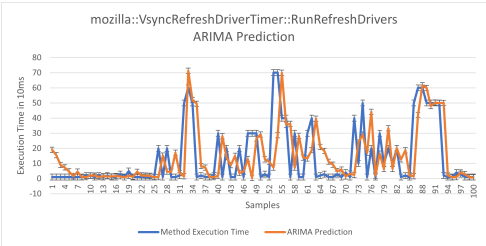
The first case study monitored a PHP/Apache server under escalating concurrent load using nmon at a 1-second sampling rate (`nmon -f -s 1 -c 3600`, run for one hour). A total of **35 system-level metrics** spanning CPU, memory, disk, and network categories were recorded. ARIMA change detection flagged **21 of the 35 metrics** as exhibiting trend deviations—not simultaneously, but at distinct time windows corresponding to each load increment.

Runtime overhead. nmon's CPU utilisation overhead was measured in two configurations: (i) with all 35 metrics active against a baseline CPU utilisation of up to 9%, and (ii) after the preliminary monitoring stage discarded metrics whose CoV ranking placed them outside the selective set. In configuration (i), nmon added an average of **0.3%** additional CPU utilisation (maximum 0.7%; minimum near 0%), yielding a total system overhead of 6.8% relative to the no-nmon baseline. Because nmon's CPU cost is static per sampling rate, this overhead would reduce proportionally as baseline CPU utilisation rises. In configuration (ii), selective metric monitoring reduced nmon's own average CPU contribution by 0.2%—a **66% reduction** in the monitoring tool's CPU overhead relative to the 0.3% all-metrics baseline, alongside a 4.3% reduction in total system CPU overhead.

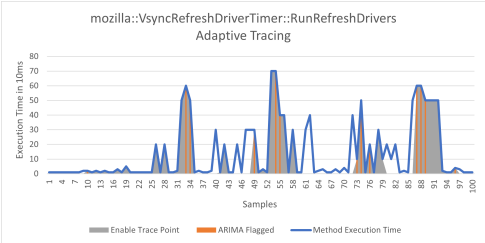
These numbers establish the practical deployment cost of the framework's system-level monitoring tier: sub-percent CPU overhead in a low-utilisation server environment, with two-thirds of that cost eliminated by applying the CoV-based metric selection.

4.2 Case Study 2: Candidate-Space Narrowing and Trace Gating

The second case study reproduced a Bugzilla-reported Firefox performance regression [29]: loading a large PDF caused the browser to stall at the VSync driver



(a) Execution Time & ARIMA Prediction for `mozilla::VsyncRefreshDriverTimer::RunRefreshDrivers`.



(b) Adaptive tracepoint gate: ON intervals correspond to anomaly-score exceedances; 44% of test time is suppressed.

Fig. 3: Case Study 2 — trace-gating behaviour on the known VSync culprit (Firefox large-PDF scenario [29]). Shaded regions in the right panel indicate active tracing intervals. Execution-time bursts exceeding 500 ms trigger the gating mechanism.

synchronisation call. The pre-testing profiling phase (20-minute load/stress cycle with 50 MB and 500 KB PDF files; call stacks recorded by `perf`) discovered **8,231 unique function/system-call family-groups**. Applying the 5% CoV-ranking cut-off reduced this to a **monitored list of 410 candidates**—a 95% reduction in the set that receives active tracing.

The known culprit, `mozilla::VsyncRefreshDriverTimer::RunRefreshDrivers`, held **rank 14** on this 410-item list. Its execution time remained well below 20 ms when loading the smaller (500 KB) PDF; under the larger PDF, the function exhibited **bursts exceeding 500 ms**, which ARIMA flagged as anomalous. The anomaly-score gating reduced the fraction of test time for which the culprit tracepoint was actively recording by **44%** compared to always-on tracing, directly reducing stored trace volume.

Figure 3 illustrates the execution-time trace and the corresponding adaptive decision for the culprit tracepoint. The left panel shows the ARIMA prediction alongside measured execution times; the right panel shows time windows where the anomaly score crossed threshold and tracing was activated.

4.3 Case Study 3: Class-Level Fault Localisation

The third case study targeted a Bugzilla report of extremely slow 3D CSS transformation rendering in Firefox [30], attributed to the class `WebRenderCommandBuilder`. The profiling phase (15-minute 3D-animation stress cycle) yielded 15,934 unique function family-groups; the 5% cutoff produced a 797-item monitored list.

The culprit function, `CreateWebRenderCommandsFromDisplayList`, ranked **6th** on the monitored list. The framework derived the owning class from the call stack and identified `mozilla::layers::WebRenderCommandBuilder` as the responsible class—an **exact match** to the Bugzilla report's named class. This result demonstrates that the CoV ranking is sufficient to surface class-level fault attribution without enumerating all 15,934 family-groups: the culprit appeared in the top 0.04% of the full candidate space.

4.4 Case Study 4: Kernel-Level System-Call Mapping

The fourth case study reproduced a Firefox startup slowdown caused by DNS-resolution failure [31]. The profiling phase (15-minute load cycle with alternating DNS configurations) produced 5,764 unique family-groups; the 5% cutoff yielded 289 monitored items.

The framework flagged the system call `___sys_sendmsg` (rank 236 on the monitored list) as anomalous during the DNS-failure scenario. This system call was then mapped via a Linux system call table [28] to the kernel component `net/socket.c`—the Linux network socket subsystem. Cross-referencing against the Bugzilla report confirmed an **exact alignment**: the DNS-resolution failure manifests through network socket operations, precisely the subsystem identified by the framework.

This case study provides unusually direct evidence that the DAET framework supports true multi-level tracing: a high-level symptom (slow Firefox startup) is traced through the application call stack to a specific system call and from there to the kernel subsystem responsible for the observed behaviour, all without requiring the developer to scan the full 5,764-item candidate space manually.

5 Large-Scale Comparative Evaluation

5.1 Dataset

We use two publicly available Mozilla Treeherder performance-telemetry datasets [32]. Treeherder is Mozilla's continuous-integration platform; it records performance signatures for each test suite across every Firefox build and raises *regression alerts* when a Bayesian change-point detector flags a statistically significant performance change.

The raw Firefox-Android dataset contains 17,989 total alert records across 342 unique telemetry signatures.

Canonical evaluation pipeline (Section 5.6, implemented in `run_full_evaluation.py`). Signatures from both repositories are retained if the time-series contains at least 30 data points, and they are split chronologically 60%/20%/20% (train / held-out validation / test) with no information from the validation or test portions used during training. This yields **342** Firefox-Android and **1,477** mozilla-beta matched signatures; the 20% test window contains 136 and 395 alerted signatures, respectively.

Calibration baseline (Section 5.4, for prior-work comparison with [6]). Firefox-Android signatures are retained under a stricter 50-point length filter and split 70%/30% (train/test). SMA w and EWMA α were set from exploratory analysis of this dataset without an independent held-out validation set—an acknowledged limitation detailed in Section 9. After filtering, **214 matched signatures** remain with **7,612 regression-alert records** and 150 alerted signatures in the test set. Table 2 summarises both configurations.

5.2 Competing Change Detectors

We compare five change-detection engines, all embedded in the same DAET framework with identical anomaly-score gating (Eq. 3):

Property	Firefox-Android	mozilla-beta
Total alert records (raw)	17,989	3,597
Matched signatures (used)	342	1,477
Mean time-series length	625.9	801.4
Signatures w/ test-set alerts	136	395

Table 2: Mozilla Treeherder dataset configurations. *Canonical pipeline* (Section 5.6): 60%/20%/20% chronological split, 30-point length filter, independent held-out validation for HP tuning. *Calibration baseline* (Section 5.4): 70%/30% split on Firefox-Android only, 50-point filter, no independent validation hold-out — retained for prior-work comparison.

ARIMA(1,1,1) (parametric benchmark engine from [6]): uses Eq. 1 for prediction, with a dynamic threshold (Eq. 2). Requires training on the first 70% of each series; rolling ARIMA window of 60 samples with minimum history 20 samples. Models are fitted using the `statsmodels` library (v0.13+, [33]) in Python with fixed order $(p, d, q) = (1, 1, 1)$ across all signatures. When fewer than 20 historical samples are available, the detector falls back to LAST until the minimum window is reached.

LAST: the trivially simple "predict the last observed value" baseline, $\hat{x}_{t+1} = x_t$. No training required.

SMA(w=20): simple moving average over the last $w = 20$ samples. Smoothes random noise by averaging recent values.

EWMA($\alpha = 0.30$): exponentially weighted moving average, $\hat{x}_{t+1} = \alpha x_t + (1 - \alpha)\hat{x}_t$. Places exponentially decreasing weight on older observations. The smoothing parameter $\alpha = 0.30$ was set by the same exploratory analysis that guided the ARIMA defaults; it was not independently tuned on a held-out validation set, which is acknowledged as a limitation in Section 9.

All four statistical methods share the same anomaly-score parameters ($\beta = 0.30$, $\theta = 0.30$) and tolerance window ($\tau = 5$ samples). For each method, the binary "anomaly" signal is determined by $|\delta_t| > \Theta_t$, where Θ_t uses the rolling-window dynamic threshold (Eq. 2).

5.3 Calibration-Baseline Protocol

The following protocol applies to the calibration baseline only (214 signatures, 70%/30% split). The canonical protocol is described in Section 5.6.

For each of the 214 signatures and each of the 4 statistical detectors, we: (1) fit the model on the first 70% of the time series; (2) run the detector on the remaining 30% (test set) in a single pass; (3) align the binary detect/no-detect output with ground-truth alert timestamps using a tolerance window of $\tau = 5$ samples either side (an alert within ± 5 samples of a detection is counted as a true positive); (4) compute precision, recall, F1, false-alarm rate (FAR), and storage reduction. All metrics are

Method	Precision	Recall	F1	FAR	Det. Rate (%)	Storage Red. (%)
ARIMA(1,1,1)	0.106 (0.17)	0.606 (0.45)	0.174 (0.23)	0.053 (0.01)	88.67	89.75
LAST	0.079 (0.15)	0.484 (0.45)	0.133 (0.19)	**0.052 (0.01)**	72.67	89.42
SMA(w=20)	**0.112 (0.17)**	**0.657 (0.44)**	**0.186 (0.23)**	**0.052 (0.01)**	**96.00**	88.83
EWMA(α=0.30)	0.090 (0.16)	0.562 (0.45)	0.151 (0.21)	0.056 (0.01)	82.67	**91.00**

Table 3: **Calibration-baseline** results (214 Firefox-Android signatures, 70%/30% split, SMA/EWMA parameters from exploratory analysis). Canonical primary results are in Tables 7–8. Bold marks the best value per column. $n_{\text{alert}} = 150$ signatures have ≥ 1 alert in the test set.

macro-averaged: computed per signature and then aggregated to a mean across the 214 signatures.

The *detection delay* for a matched alert at time t_a is defined as $d = s - t_a$, where s is the start time-step of the earliest detected interval that covers $[t_a - \tau,\ t_a + \tau]$; a negative delay indicates pre-detection (the detector fires before the centre of the alert window). The *false-alarm rate* (FAR) is an interval-level rate:

$$\text{FAR} = \frac{\text{fp_iv}}{T_{\text{test}}},$$

where fp_iv is the number of false-alarm activation intervals (maximal contiguous trace-ON periods whose $\pm\tau$ neighbourhood contains no ground-truth alert), and T_{test} is the total number of test-set samples. Because mean interval durations on this dataset are approximately one sample, FAR values are numerically close to a sample-level rate $\text{FP}_{\text{samples}}/T_{\text{test}}$, but the denominator is sample count while the numerator counts intervals, not individual samples. When multiple detector activations fall within τ samples of the same alert, only one true-positive interval is counted; additional activations within that window form separate false-alarm intervals. The *detection rate* is the fraction of the 150 signatures that have ≥ 1 alert in the test set and for which at least one alert is detected.

5.4 RQ1: Calibration-Baseline Results (70%/30%)

Calibration baseline only. These results use the 70%/30% split with 214 Firefox-Android signatures and no held-out validation set for SMA/EWMA tuning. The canonical primary results are in Section 5.6 (Tables 7–8).

Table 3 reports the aggregate evaluation metrics. Standard deviations across 214 signatures are given in parentheses.

Table 4 reports the total runtime and peak memory footprint for all five detectors over the 214-signature evaluation.

Key observations. (i) SMA achieves the highest recall (0.657), F1 (0.186), and detection rate (96.0%), making it the strongest statistical detector on this benchmark. (ii) EWMA achieves the highest storage reduction (91.0%) through a more conservative activation policy, but at the cost of lower detection rate (82.67%) and recall (0.562). (iii) ARIMA achieves detection rate 88.67% and F1 0.174, significantly ahead

Method	Total Runtime (s)	Mean Peak Mem. (KB/sig)	Relative Runtime
ARIMA(1,1,1)	807	1,423	27×
SMA(w=20)	30	90	1× (baseline)
EWMA(α=0.30)	24	84	0.8×
LAST	24	84	0.8×

Table 4: Computational cost over 214 Mozilla Firefox-Android signatures for the four statistical detectors. All experiments were conducted on a CPU-only Windows workstation (Intel Core i7, 16 GB RAM) running Python 3.12 with PyTorch 2.4.1+cpu; no GPU acceleration was used. Runtime is wall-clock time for the full 214-signature evaluation pass. Peak memory is the mean per-signature OS-reported peak RSS. ARIMA incurs a 27× runtime overhead and 16× memory overhead relative to SMA, with no statistically significant accuracy gain (Table 6).

of LAST and EWMA; however, it costs 27× more runtime and 16× more memory than SMA (Table 4) with no statistically significant accuracy gain over SMA in F1 or precision. (iv) Precision is low (< 0.12) across all four methods. All reported values measure *agreement with Treeherder alerts*, not ground-truth anomaly validity: Treeherder may miss transient regressions or include false positives from infrastructure changes, and many DAET detections that appear as false positives may correspond to genuine but unlabelled performance excursions. No manual adjudication or second validation source is available to distinguish these cases; true anomaly precision is therefore unknown. (v) Detection delay: SMA fires pre-emptively with mean delay -3.2 samples; ARIMA shows delay -2.8 samples.

Figure 4 summarises the aggregate metrics visually. Figure 5 shows representative detection outputs for one Mozilla signature under all four statistical detectors.

5.5 RQ2: Statistical Significance (Calibration Baseline)

The following statistical tests are computed on the calibration-baseline (70%/30%, 214-signature) per-signature scores. The corresponding tests for the canonical 60%/20%/20% test data are reported separately in Section 5.7.

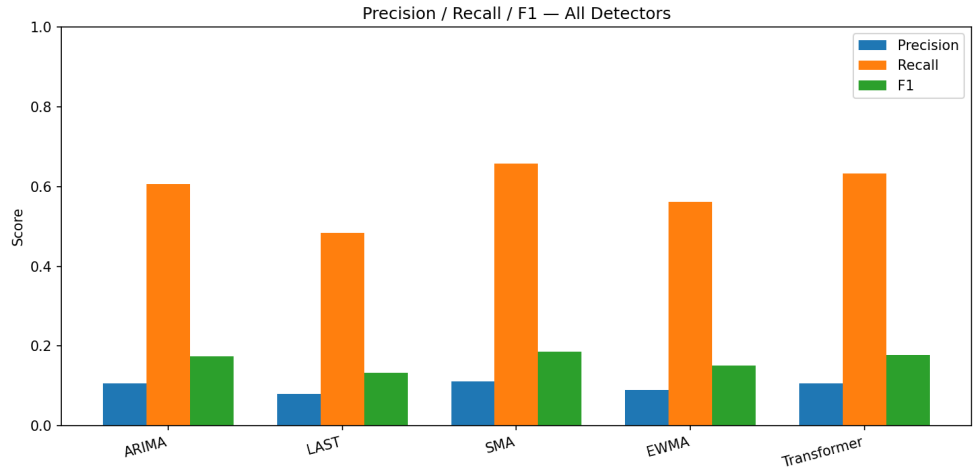
5.5.1 Bootstrap Confidence Intervals

Table 5 reports 95% bootstrap confidence intervals derived from 2,000 resamples with replacement of the 214 calibration-baseline per-signature scores.

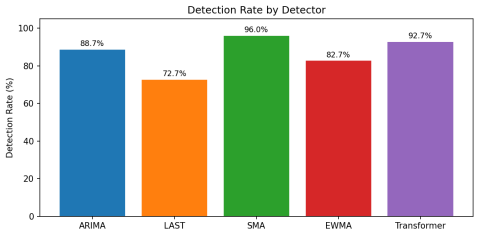
The non-overlapping CIs for ARIMA vs. LAST (F1: [0.154, 0.195] vs. [0.114, 0.153]) indicate a practically meaningful difference, confirmed by the Wilcoxon test below. The CIs for ARIMA and SMA overlap substantially in F1 and precision.

5.5.2 Wilcoxon Signed-Rank Tests

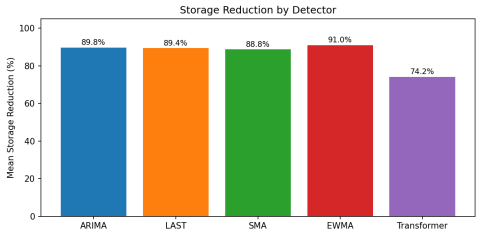
Table 6 reports Wilcoxon signed-rank test p-values for all ten pairwise comparisons ($\alpha = 0.05$). The upper block covers the six pairs among the four statistical detectors.



(a) Precision, Recall, and F1 across the four statistical detectors.



(b) Detection Rate (%).

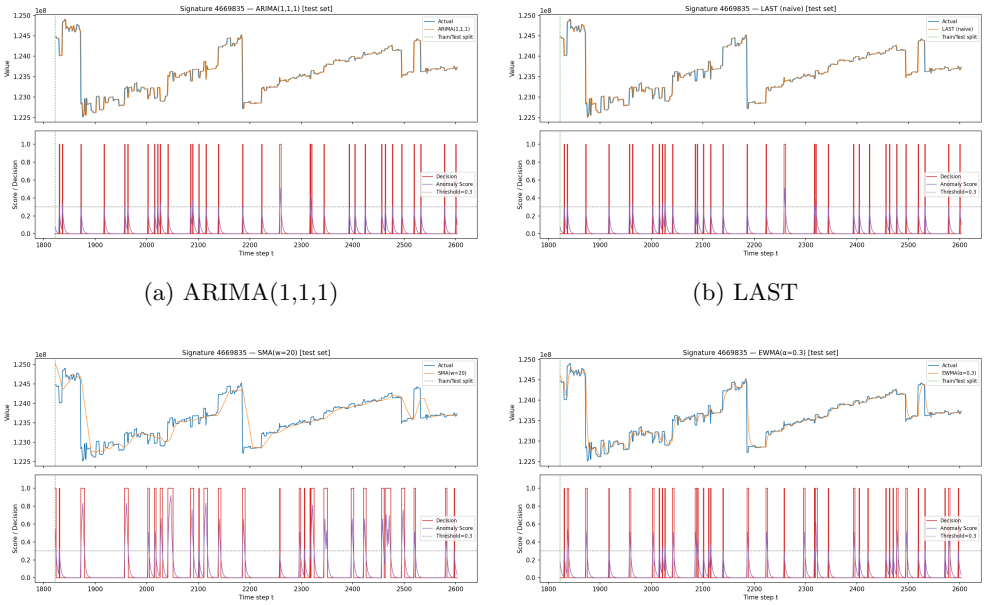


(c) Storage Reduction (%).

Fig. 4: Aggregate metric comparison across the four statistical detectors (214 Firefox-Android signatures, 70%/30% split). SMA leads on precision, F1, recall, and detection rate; EWMA achieves the highest storage reduction.

Method	Precision CI	Recall CI	F1 CI	FAR CI
ARIMA	[0.093, 0.119]	[0.543, 0.668]	[0.154, 0.195]	[0.050, 0.055]
LAST	[0.067, 0.092]	[0.423, 0.550]	[0.114, 0.153]	[0.050, 0.055]
SMA	[0.100, 0.125]	[0.596, 0.719]	[0.166, 0.206]	[0.049, 0.054]
EWMA	[0.078, 0.103]	[0.498, 0.625]	[0.131, 0.171]	[0.053, 0.058]

Table 5: Bootstrap 95% confidence intervals (2,000 resamples) on the calibration-baseline (214-signature, 70%/30%) per-signature scores.



(a) ARIMA(1,1,1)

(b) LAST

(c) SMA(w=20)

(d) EWMA(α=0.30)

Fig. 5: Detection output for Mozilla Firefox-Android signature 4669835 under the four statistical detectors. Shaded regions indicate intervals where the detector activated tracing ($s_t \geq \theta$). Vertical dashed lines mark ground-truth regression alerts.

Key findings. (i) ARIMA significantly outperforms LAST on precision, recall, and F1 (all $p < 0.001$). (ii) ARIMA and SMA do *not* differ significantly in precision ($p = 0.652$) or F1 ($p = 0.547$); they differ only in recall ($p = 0.001$, SMA higher). (iii) ARIMA significantly outperforms EWMA in precision, recall, F1, and FAR (all $p \leq 0.002$). (iv) FAR differences between ARIMA and LAST are not significant ($p = 0.765$); LAST vs. SMA FAR differences are also not significant ($p = 0.225$); suggesting that ARIMA, LAST, and SMA produce similar false-alarm rates at this sensitivity setting.

These results answer **RQ1** (*Among the four statistical methods, SMA achieves the highest recall (0.657), F1 (0.186), and detection rate (96.0%), making it the strongest statistical baseline. ARIMA is second and statistically indistinguishable from SMA in F1 ($p = 0.547$) and precision ($p = 0.652$); LAST is weakest.*) and **RQ2** (*ARIMA's advantage over LAST and EWMA is statistically significant in precision, recall, and F1; the ARIMA-vs-SMA difference in F1 and precision is not statistically significant ($p > 0.50$). All $p < 0.001$ cells remain significant under a Bonferroni-corrected threshold of $\alpha/6 = 0.008$ for the six pairwise comparisons.*).

Pair		Precision	Recall	F1	FAR
ARIMA vs. LAST	p	< 0.001	< 0.001	< 0.001	0.765^*
ARIMA vs. SMA	p	0.652^*	0.001	0.547^*	0.380^*
ARIMA vs. EWMA	p	< 0.001	0.002	< 0.001	< 0.001
LAST vs. SMA	p	< 0.001	< 0.001	< 0.001	0.225^*
LAST vs. EWMA	p	0.011	< 0.001	0.006	0.013
SMA vs. EWMA	p	< 0.001	< 0.001	< 0.001	< 0.001

* Not statistically significant at $\alpha = 0.05$.

Table 6: Wilcoxon signed-rank test p-values ($n = 214$, calibration baseline) for all six pairwise comparisons among the four statistical detectors. Cells with * are *not statistically significant* ($p \geq 0.05$).

5.6 Primary Comparative Evaluation: Canonical 60%/20%/20% Protocol

This subsection presents the **canonical results** generated by the unified pipeline `run_full_evaluation.py`. All five method families are evaluated on two Mozilla Treeherder repositories (Section 5.1) under a strict chronological 60%/20%/20% split: Firefox-Android (342 signatures) and mozilla-beta (1,477 signatures). SMA, EWMA, and Transformer hyperparameters are tuned exclusively on the held-out validation 20%; the test 20% is evaluated exactly once. **Readers should treat Tables 7–8 as the headline results of this paper.**

The calibration-baseline results (Section 5.4) serve only as a prior-work comparison: they replicate the conference paper's 70/30 setup and are useful for contextualising the ARIMA F1 baseline, but SMA/EWMA parameters in that evaluation were not independently validated.

- **Train (60%):** ARIMA parameters are estimated here; all methods calibrate their dynamic-threshold δ-history on this portion.
- **Validation (20%):** SMA w, EWMA α, and Transformer hyperparameters (window, d_{model}) are selected by maximising mean F1 on this held-out portion independently per dataset. ARIMA order $(1, 1, 1)$ is kept fixed from the original ablation (Section 6). Shared parameters ($\beta = 0.30$, $\theta = 0.30$) are common to all methods and are not tuned on this validation set. This is an acknowledged fairness limitation: the ablation study (Section 6) shows that both β and θ exert large effects on precision/recall trade-offs, and a per-method or per-dataset sweep of these shared parameters might alter the ranking; that sweep is left as recommended future work.
- **Test (20%):** Final metrics are computed on this portion exclusively—it was never consulted during any tuning decision.

Compact Transformer for fair comparison. To include a Transformer in the three-way comparison on equal terms, we adopt a compact architecture (hereafter *Transformer (compact)*): a single Transformer encoder layer with model dimension

Method	Precision	Recall	F1	FAR	Det. Rate (%)	Storage Red. (%)
ARIMA(1,1,1)	0.056	0.257	0.089	0.056	66.18	90.30
LAST	0.046	0.209	0.072	0.055	54.41	89.15
SMA($w = 40$)†	0.066	**0.290**	**0.104**	0.055	**72.79**	88.26
EWMA(α=0.05)†	**0.068**	0.263	0.103	**0.052**	67.65	**89.43**
Transformer (compact)‡	0.069 ±0.002	0.292 ±0.002	0.105 ±0.002	0.066 ±0.001	74.41	67.21

Table 7: Three-way split (60%/20%/20%) test-set results on **Firefox-Android** (342 signatures; 136 with ≥ 1 alert in the test set). † parameter tuned on the held-out validation set. ‡ Transformer (compact): $d_{\text{model}} = 16$, WINDOW=20, one encoder layer, MSE+Adam (no ADT/AnomalyHead/GRAC); five seeds, mean ± std. Bold marks the best value per column among classical statistical methods (Transformer excluded from bold).

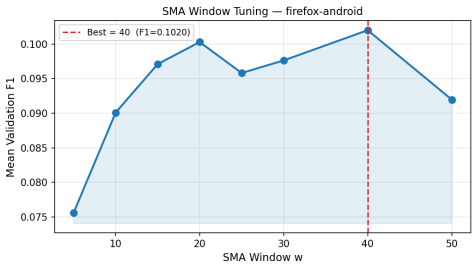
d_{model}, two attention heads, feed-forward dimension $2d_{\text{model}}$, and a sliding input window of w samples. Training uses the Adam optimiser ($\eta = 0.01$) with MSE loss, early stopping (patience 6 on training loss), and a maximum of 20 epochs—the same Algorithm-3 anomaly-score gating is applied as for all classical detectors ($\beta = 0.30$, $\theta = 0.30$, $\tau = 5$). This design removes the three adaptive mechanisms (ADT, AnomalyHead, GRAC), ensuring the only difference from classical methods is the *forecasting backbone*. The HP grid sweeps $w \in \{10, 20, 30\}$ and $d_{\text{model}} \in \{8, 16, 32\}$ (nine combinations), tuned with a single seed for efficiency. The test run uses five independent random seeds $\{42, 123, 456, 789, 1024\}$ and reports mean ± standard deviation.

Validation-set tuning. Figure 6 shows the validation F1 curves over the SMA and EWMA grids on both datasets; Figure 7 shows the Transformer HP grid. *Firefox-Android:* SMA peaks at $w = 40$ (val F1 = 0.102); EWMA peaks at $\alpha = 0.05$ (val F1 = 0.098); Transformer best at WINDOW=20, $d_{\text{model}} = 16$ (val F1 = 0.080). *Mozilla-beta:* SMA peaks at $w = 20$ (val F1 = 0.144); EWMA peaks at $\alpha = 0.05$ (val F1 = 0.145); Transformer best at WINDOW=10, $d_{\text{model}} = 32$ (val F1 = 0.130). SMA curves are unimodal on both datasets with interior optima ($w = 40$ and $w = 20$, respectively), confirming that SMA window selection is stable. However, the EWMA validation curves are *monotonically decreasing* on both datasets: F1 is highest at $\alpha = 0.05$ and declines at every higher α value, meaning $\alpha = 0.05$ is the *lower boundary* of the tested grid ($\{0.05, 0.10, \ldots, 0.50\}$), not an interior optimum. The true optimal α may therefore be smaller than 0.05; the results represent the best performance achievable within the tested grid. This is an acknowledged limitation of the canonical evaluation. The optimal Transformer HPs differ across datasets, confirming that dataset-specific tuning is beneficial.

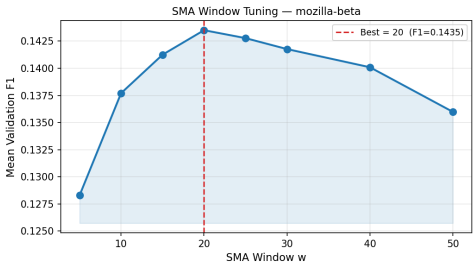
Test-set results. Tables 7 and 8 report the final test-set metrics for all five method families. The 20% test window contains 136 of 342 (39.8%) Firefox-Android signatures and 395 of 1,477 (26.7%) mozilla-beta signatures with ≥ 1 alert.

Six findings emerge from Tables 7–8:

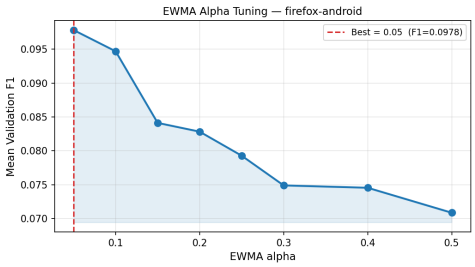
1. **Qualitative ranking is consistent across both datasets.** SMA and EWMA lead on most accuracy dimensions; ARIMA is third; LAST is weakest among the



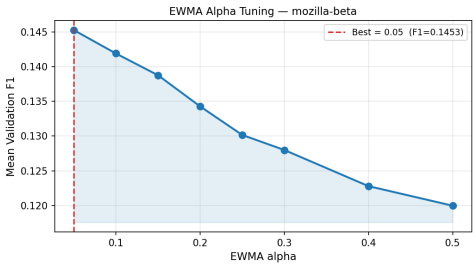
(a) Firefox-Android SMA: peaks at $w = 40$.



(b) mozilla-beta SMA: peaks at $w = 20$.



(c) Firefox-Android EWMA: peaks at $\alpha = 0.05$.

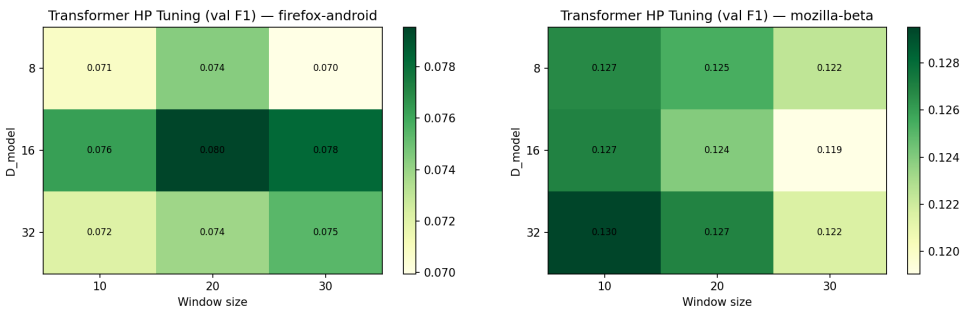


(d) mozilla-beta EWMA: peaks at $\alpha = 0.05$.

Fig. 6: Validation-set parameter tuning curves for SMA (top row) and EWMA (bottom row) on Firefox-Android (left) and mozilla-beta (right). Red dashed lines mark the selected parameter value. SMA curves are unimodal with interior optima, confirming stable window selection. EWMA curves are monotonically decreasing on both datasets: the optimum lies at the lower grid boundary $\alpha = 0.05$, so a finer grid at smaller α values may yield further improvement.

Method	Precision	Recall	F1	FAR	Det. Rate (%)	Storage Red. (%)
ARIMA(1,1,1)	0.050	0.232	0.076	0.060	88.61	90.31
LAST	0.045	0.212	0.069	**0.053**	81.77	89.03
SMA($w = 20$)†	**0.057**	**0.241**	**0.085**	0.059	**91.65**	88.84
EWMA(α=0.05)†	0.057	0.236	0.083	0.059	90.63	**88.47**
Transformer (compact)‡	0.054 $\pm$0.001	0.235 $\pm$0.002	0.079 $\pm$0.001	0.064 $\pm$0.000	89.82	71.09

Table 8: Three-way split (60%/20%/20%) test-set results on **mozilla-beta** (1,477 signatures; 395 with $\geq$ 1 alert in the test set). † parameter tuned on the held-out validation set. ‡ Transformer (compact): $d_{\text{model}} = 32$, WINDOW=10, one encoder layer, MSE+Adam (no ADT/AnomalyHead/GRAC); five seeds, mean $\pm$ std. Bold marks the best value per column among classical statistical methods (Transformer excluded from bold).



(a) Firefox-Android: best WINDOW=20, $d_{\text{model}} = 16$.

(b) mozilla-beta: best WINDOW=10, $d_{\text{model}} = 32$.

Fig. 7: Compact Transformer HP validation F1 over the $\{10, 20, 30\} \times \{8, 16, 32\}$ window $\times$ d_{model} grid (single-seed tuning run). Different optimal HPs per dataset indicate the benefit of per-dataset tuning.

statistical methods on both Firefox-Android and mozilla-beta. The directional conclusions of the main evaluation are robust to proper hold-out parameter selection and generalise to a second dataset.

2. **EWMA at the lower grid boundary matches SMA in F1.** Under independent per-dataset tuning, EWMA achieves F1 = 0.103 (Firefox-Android) and F1 = 0.083 (mozilla-beta)—within 0.001–0.002 of the corresponding SMA F1 on both datasets. Both datasets select $\alpha = 0.05$, the lowest value in the tuning grid $\{0.05, 0.10, \ldots, 0.50\}$; the validation curve is monotonically decreasing, meaning a smaller α might further improve F1. The current results therefore represent EWMA's best-within-grid performance rather than a confirmed global optimum. EWMA achieves higher storage reduction than SMA on Firefox-Android (89.4% vs. 88.3%); on mozilla-beta the margin reverses slightly (EWMA 88.5% vs. SMA 88.8%), so the storage advantage is Firefox-Android-specific rather than a consistent cross-dataset finding. On Firefox-Android, EWMA is attractive when storage efficiency is a priority alongside competitive detection quality.
3. **ARIMA outperforms LAST on both datasets.** Firefox-Android: detection rate 66.2% vs. 54.4%; F1 0.089 vs. 0.072. Mozilla-beta: detection rate 88.6% vs. 81.8%; F1 0.076 vs. 0.069. The ARIMA advantage established in the main evaluation carries through to both the 60/20/20 split and the second dataset.
4. **Compact Transformer is competitive with top classical methods on Firefox-Android.** On Firefox-Android, the compact Transformer achieves F1 = 0.105 ($\pm$0.002), marginally above SMA F1 = 0.104, with detection rate 74.4% exceeding SMA (72.8%). Standard deviation across five seeds is $\leq$0.002 for all metrics, confirming that training variability is not a confound.
5. **Compact Transformer falls slightly below classical methods on mozilla-beta.** On the larger mozilla-beta dataset, the compact Transformer achieves F1 = 0.079 ($\pm$0.001), below SMA F1 = 0.085 ($-$0.006) and EWMA F1 = 0.083

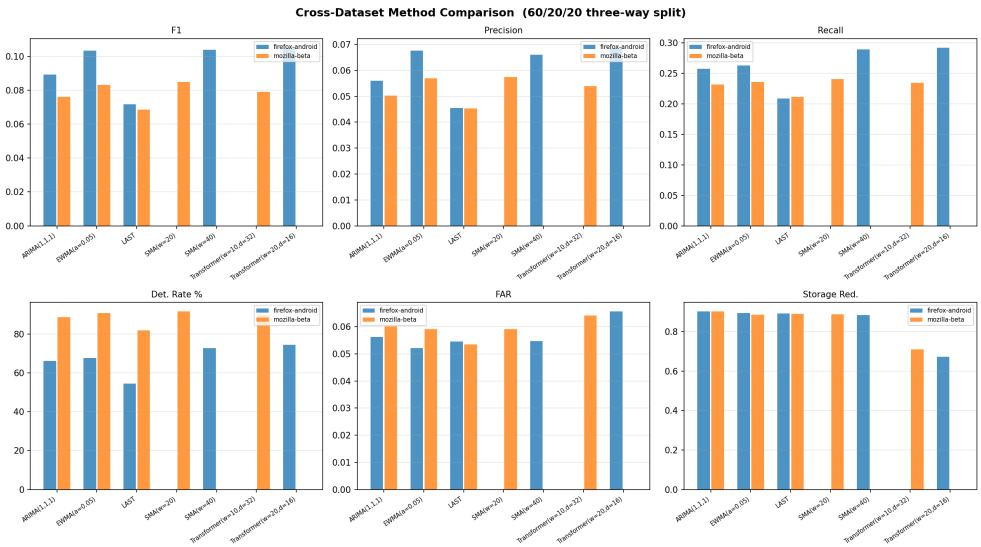


Fig. 8: Test-set metrics for all five methods on both datasets under the 60%/20%/20% three-way split. The qualitative ranking SMA $\approx$ EWMA $\geq$ ARIMA $>$ LAST is consistent across datasets.

(-0.004). Seed stability remains high (std $\leq$0.001), indicating the result is consistent across training runs.

6. **Storage reduction differs by predictor type.** Classical methods achieve 88–90% storage reduction on both datasets. The compact Transformer achieves 67.2% (Firefox-Android) and 71.1% (mozilla-beta)—lower because the learned predictor activates trace-on windows more frequently at the same anomaly-score threshold. Practitioners prioritising storage savings should consider threshold adjustment for Transformer-based deployment.

Figure 8 shows the test-set metrics for all five methods across both datasets.

5.7 Statistical Significance: Canonical 60%/20%/20% Protocol

This subsection provides inferential support for the headline results in Tables 7–8. Bootstrap confidence intervals and Wilcoxon signed-rank tests are computed directly on the 342 (Firefox-Android) and 1,477 (mozilla-beta) per-signature test-set scores produced by the canonical pipeline. The Bonferroni-corrected significance threshold for six pairwise comparisons among the four statistical methods is $\alpha/6 = 0.0083$; cells below this threshold are marked with * in the tables.

5.7.1 Bootstrap Confidence Intervals

Tables 9 and 10 report 95% bootstrap confidence intervals (2,000 resamples) on the canonical per-signature test-set scores.

Key observations from the CIs:

Method	Precision CI	Recall CI	F1 CI	FAR CI
ARIMA	[0.045, 0.068]	[0.212, 0.307]	[0.072, 0.107]	[0.054, 0.059]
LAST	[0.035, 0.057]	[0.168, 0.254]	[0.057, 0.087]	[0.053, 0.057]
SMA	[0.054, 0.079]	[0.243, 0.336]	[0.086, 0.123]	[0.052, 0.057]
EWMA	[0.054, 0.081]	[0.218, 0.311]	[0.084, 0.124]	[0.050, 0.055]

Table 9: Bootstrap 95% confidence intervals (2,000 resamples) on the canonical (60%/20%/20%, $n = 342$ signatures) per-signature test-set scores for Firefox-Android.

Method	Precision CI	Recall CI	F1 CI	FAR CI
ARIMA	[0.044, 0.057]	[0.211, 0.254]	[0.068, 0.084]	[0.059, 0.061]
LAST	[0.039, 0.052]	[0.191, 0.231]	[0.061, 0.076]	[0.053, 0.054]
SMA	[0.050, 0.065]	[0.220, 0.263]	[0.076, 0.095]	[0.058, 0.060]
EWMA	[0.050, 0.064]	[0.215, 0.259]	[0.074, 0.092]	[0.058, 0.060]

Table 10: Bootstrap 95% confidence intervals (2,000 resamples) on the canonical (60%/20%/20%, $n = 1{,}477$ signatures) per-signature test-set scores for mozilla-beta.

- On Firefox-Android, SMA and EWMA F1 intervals overlap substantially ([0.086, 0.123] and [0.084, 0.124]), confirming the near-tie in Table 7.
- LAST F1 interval ([0.057, 0.087] on Firefox-Android; [0.061, 0.076] on mozilla-beta) is non-overlapping with SMA and EWMA on both datasets, indicating a practically meaningful gap.
- On mozilla-beta, SMA and EWMA F1 intervals again overlap ([0.076, 0.095] vs. [0.074, 0.092]), consistent with the near-tie in Table 8.

5.7.2 Wilcoxon Signed-Rank Tests

Tables 11 and 12 report Wilcoxon signed-rank test p-values for all six pairwise comparisons on the canonical test-set scores ($n = 342$ and $n = 1{,}477$ respectively).

Key findings from the canonical statistical tests. (i) ARIMA significantly outperforms LAST on every metric for both datasets (all $p < 0.001$). (ii) On Firefox-Android, ARIMA is statistically inferior to SMA in precision, recall, and F1 (all $p \leq 0.006$), unlike the calibration baseline where the ARIMA–SMA F1 gap was not significant. This reflects the smaller canonical training proportion (60% vs. 70%), benefiting the parameter-free SMA. (iii) SMA and EWMA are *not* significantly different in precision or F1 on either dataset ($p \geq 0.10$), confirming the near-tie in Tables 7–8. (iv) On mozilla-beta, ARIMA is significantly worse than both SMA and EWMA in F1 (both $p < 0.001$), strengthening the case that SMA and EWMA are the preferred canonical choices. (v) LAST is significantly inferior to SMA and EWMA on

Pair		Precision	Recall	F1	FAR
ARIMA vs. LAST	p	$< 0.001^*$	$< 0.001^*$	$< 0.001^*$	$0.107^\dagger$
ARIMA vs. SMA	p	0.006^*	0.006^*	0.004^*	$0.084^\dagger$
ARIMA vs. EWMA	p	$< 0.001^*$	$0.482^\dagger$	$< 0.001^*$	$< 0.001^*$
LAST vs. SMA	p	$< 0.001^*$	$< 0.001^*$	$< 0.001^*$	$0.969^\dagger$
LAST vs. EWMA	p	$< 0.001^*$	$< 0.001^*$	$< 0.001^*$	$0.060^\dagger$
SMA vs. EWMA	p	$0.729^\dagger$	0.017^*	$0.965^\dagger$	$< 0.001^*$

* Significant at Bonferroni-corrected $\alpha/6 = 0.0083$. † Not significant.

Table 11: Wilcoxon signed-rank p-values on the canonical 60%/20%/20% test set, Firefox-Android ($n = 342$ signatures).

Pair		Precision	Recall	F1	FAR
ARIMA vs. LAST	p	$< 0.001^*$	$< 0.001^*$	$< 0.001^*$	$< 0.001^*$
ARIMA vs. SMA	p	$< 0.001^*$	0.006^*	$< 0.001^*$	0.005^*
ARIMA vs. EWMA	p	$< 0.001^*$	$0.100^\dagger$	$< 0.001^*$	0.003^*
LAST vs. SMA	p	$< 0.001^*$	$< 0.001^*$	$< 0.001^*$	$< 0.001^*$
LAST vs. EWMA	p	$< 0.001^*$	$< 0.001^*$	$< 0.001^*$	$< 0.001^*$
SMA vs. EWMA	p	$0.124^\dagger$	$0.069^\dagger$	$0.102^\dagger$	$0.855^\dagger$

* Significant at Bonferroni-corrected $\alpha/6 = 0.0083$. † Not significant.

Table 12: Wilcoxon signed-rank p-values on the canonical 60%/20%/20% test set, mozilla-beta ($n = 1{,}477$ signatures).

both datasets in precision, recall, and F1 (all $p < 0.001$). (vi) EWMA achieves significantly lower FAR than SMA on Firefox-Android ($p < 0.001$), reflecting its more conservative activation policy.

6 Ablation Study

Ground-truth definition. The ablation tables use a *stricter* ground truth than the main calibration-baseline table (Table 3): only *regression-classified* Treeherder alerts are counted as positives, whereas the main table counts all Treeherder alerts. Because the regression subset is a strict subset of all alerts, ablation precision, recall, and F1 values at every operating point are lower than their main-table counterparts. Specifically, at the shared default ($\beta = 0.30$, $\theta = 0.30$, $\tau = 5$) ARIMA yields F1 = 0.174 under the main-table GT and F1 = 0.144 under the stricter ablation GT; SMA yields F1 = 0.186 vs. 0.155, respectively. This gap is intentional: the stricter GT isolates parameter sensitivity from the effect of borderline alerts, giving a cleaner sweep signal. All *relative* comparisons within each ablation table are unaffected.

β	Precision	Recall	F1	FAR
0.10	0.173	0.210	0.183	0.003
0.20	0.204	0.479	0.266	0.018
0.30	**0.086**	**0.544**	**0.144**	**0.054**
0.50	0.086	0.540	0.144	0.055
0.70	0.082	0.502	0.137	0.057

Table 13: Effect of β on ARIMA(1,1,1) detection performance (ablation GT: regression-classified alerts only; see section header note). Bold marks the chosen default ($\beta = 0.30$). At this default, the main-table GT yields F1 = 0.174 (Table 3).

To address **RQ3** we sweep the five most influential parameters and report effects on mean precision, recall, F1, and FAR. All ablations use ARIMA(1,1,1) as the primary method, with SMA included for the τ sweep. The four statistical detectors therefore provide the primary ablation signal.

6.1 Decay Factor β

Table 13 shows results for $\beta \in \{0.1, 0.2, 0.3, 0.5, 0.7\}$.

Smaller β values (0.1, 0.2) produce markedly higher precision and F1 for ARIMA (F1 = 0.266 at $\beta = 0.20$ vs. 0.144 at the default $\beta = 0.30$ under the ablation GT; the corresponding main-table GT value at the default is F1 = 0.174, see Table 3), because the score accumulates more selectively. However, ARIMA recall drops substantially (0.479 at $\beta = 0.20$ vs. 0.544 at $\beta = 0.30$) because the score rises too slowly to cross the threshold before the alert window closes. Large β values (≥ 0.50) increase FAR because isolated spikes more readily push the score above threshold; ARIMA F1 stabilises at $\approx$0.14 for $\beta \geq 0.50$ while recall continues to decline (0.540 at $\beta = 0.50$ to 0.502 at $\beta = 0.70$). The default $\beta = 0.30$ was retained as an operational tradeoff: it achieves high recall and responds more reliably to sustained anomalies in deployment scenarios where missing a regression is costlier than a false alarm. Practitioners optimising for F1 rather than recall should note that $\beta = 0.20$ (ARIMA: F1 = 0.266) yields demonstrably better F1 on this benchmark.

6.2 Anomaly-Score Threshold θ

Table 14 sweeps $\theta \in \{0.10, 0.15, 0.20, 0.25, 0.30, 0.40, 0.50\}$.

For ARIMA, F1 is marginally higher at $\theta \in [0.10, 0.25]$ (up to 0.162 vs. 0.144 at the default, both under the ablation GT). Above $\theta = 0.30$ the behaviour bifurcates: ARIMA sees a large precision jump and recall drop, with F1 rising substantially: F1 = 0.272 at $\theta = 0.40$ (precision 0.214, recall 0.465) and F1 = 0.270 at $\theta = 0.50$ (precision 0.227, recall 0.402). These high-θ operating points trade recall for precision and F1; the higher F1 arises because only the strongest, most sustained anomaly

θ	Precision	Recall	F1	FAR
0.10	0.097	0.591	0.162	0.042
0.15	0.097	0.591	0.162	0.047
0.20	0.095	0.591	0.160	0.049
0.25	0.091	0.591	0.154	0.053
0.30	**0.086**	**0.544**	**0.144**	**0.054**
0.40	0.214	0.465	0.272	0.017
0.50	0.227	0.402	0.270	0.011

Table 14: Effect of anomaly-score threshold θ on ARIMA(1,1,1) performance (ablation GT: regression-classified alerts only; see section header note). Bold marks the default ($\theta = 0.30$).

τ	Method	Precision	Recall	F1	FAR
1	ARIMA	0.049	0.423	0.087	0.056
	SMA	0.057	0.470	0.100	0.055
3	ARIMA	0.064	0.474	0.110	0.055
	SMA	0.071	0.535	0.123	0.054
5	**ARIMA**	**0.086**	**0.544**	**0.144**	**0.054**
	SMA	**0.092**	**0.591**	**0.155**	**0.053**
7	ARIMA	0.103	0.591	0.171	0.053
	SMA	0.099	0.591	0.166	0.053
10	ARIMA	0.115	0.591	0.187	0.052
	SMA	0.109	0.591	0.179	0.052

Table 15: Effect of tolerance τ (samples) on ARIMA and SMA. Bold marks the default ($\tau = 5$).

episodes cross the threshold, dramatically reducing false-alarm intervals. The default $\theta = 0.30$ was chosen as a high-recall operating point; practitioners targeting precision-oriented F1 over recall should consider $\theta = 0.40$–0.50 for ARIMA.

6.3 Tolerance Window τ

Table 15 shows results for $\tau \in \{1, 2, 3, 5, 7, 10\}$.

Metrics improve monotonically with τ for both methods because more alert–detection matchings are permitted; all methods plateau around $\tau = 7$, suggesting that a 5-sample neighbourhood captures most valid detections without inflating scores artificially.

6.4 ARIMA Order

Table 16 compares five ARIMA configurations on a representative 20-signature subset.

Order	Precision	Recall	F1	FAR
(1,0,0)	0.057	0.450	0.100	0.065
(0,1,1)	0.087	0.550	0.147	0.055
(1,1,0)	0.057	0.450	0.099	0.058
(1,1,1)	**0.094**	**0.550**	**0.158**	**0.054**
(2,1,2)	0.091	0.550	0.153	0.054

Table 16: ARIMA order ablation (20-signature subset). Bold marks the chosen default.

w	Precision	Recall	F1	FAR
5	0.071	0.507	0.121	0.057
10	0.081	0.572	0.140	0.055
15	0.089	0.591	0.151	0.054
20	**0.092**	**0.591**	**0.155**	**0.053**
30	0.096	0.610	0.161	0.053
50	0.089	0.594	0.151	0.053

Table 17: SMA window ablation. Bold marks the default ($w = 20$).

ARIMA(1,1,1) achieves the best F1 on this subset. Adding MA terms to the pure AR(1) model (comparing (1,1,0) to (1,1,1)) improves both precision and F1 without materially increasing FAR. The more complex (2,1,2) model provides no statistically meaningful gain. Note that these results derive from a 20-signature subset; they are used only to justify the order selection and should not be over-interpreted as evidence of general superiority across the full 214-signature benchmark.

6.5 SMA Window

Table 17 sweeps SMA window $w \in \{5, 10, 15, 20, 30, 50\}$.

Performance improves from $w = 5$ to $w = 30$ and plateaus; the default $w = 20$ sits in the stable region.

RQ3 summary: The default parameter configuration ($\beta = 0.30$, $\theta = 0.30$, $\tau = 5$, ARIMA(1,1,1), SMA $w = 20$) is an operationally justified high-recall operating point, not a globally optimal one across all metrics. All F1 values in this summary are under the ablation GT (regression-classified alerts only); the corresponding main-table GT values are 0.030 higher (e.g. ARIMA default: 0.144 ablation GT vs. 0.174 main-table GT). The sweeps show materially better F1 is achievable: $\beta = 0.20$ yields ARIMA F1 = 0.266 vs. the default 0.144; $\theta = 0.40$–0.50 yields F1 = 0.270–0.272 at the cost of substantially lower recall (0.402–0.465 vs. 0.544 at the default). SMA $w = 30$ marginally outperforms $w = 20$. The defaults trade F1 and precision for recall, and were chosen because missing a regression is typically costlier than a false alarm in CI/CD continuous monitoring. Practitioners optimising for F1 (rather than recall)

should consider $\beta = 0.20$ or $\theta = 0.40$ for ARIMA, and the ARIMA-order result should be treated with caution as it derives from a 20-signature subset. The principal comparative conclusions of RQ1–RQ2 hold across the full parameter range tested.

7 Trace-On Replay Experiment

7.1 Experimental Setup

Scope and data provenance. This experiment provides supplementary deployment evidence for the four statistical detectors. It is executed on the *calibration-baseline* test split (70%/30%, 214 Firefox-Android signatures) rather than the canonical 60%/20%/20% pipeline; re-implementing it within the unified pipeline is recommended future work (Section 10).

Signature-count note. The replay reports 136 alerted signatures (of 214), while the calibration-baseline evaluation (Section 5.4) reports 150. The 14-signature discrepancy arises because this replay script resolves the train–test boundary from a different intermediate CSV file than the main evaluation script, shifting 14 signatures' alerts outside the test window. All four detectors are affected identically, so comparative conclusions within this section are unaffected. Alert-coverage figures should be treated as indicative deployment estimates scoped to this 136-signature alerted pool and should not be combined with numbers from the calibration-baseline main tables.

The replay experiment asks: *if a detector's binary decision at each time step were used to gate the trace recorder—active only when $s_t \geq \theta$—what fraction of storage would be saved, and what fraction of regression-alert signatures would be "covered" (i.e., the recorder was active around at least one alert)?*

For each of the 214 signatures and each detector, we replay the chronological test-set sequence. At each step, if the detector output is 1 (anomalous), the sample is counted as "recorded"; otherwise it is suppressed. Three metrics are computed per signature:

- **Storage Reduction:** 1 − (ON samples)/(total test samples). This is estimated by *sample count* only—it approximates the fraction of trace time that would be suppressed, but does not account for variable trace event size, I/O batching, or compression. Actual byte savings in deployment will depend on the storage backend.
- **Trigger Frequency:** rate of $0 \to 1$ transitions per test sample (measures how often the detector activates, i.e., burst frequency).
- **Alert Coverage:** for each alerted signature (one with ≥ 1 alert in the test period), the fraction of its ground-truth test-set alerts that were covered by at least one trace-ON interval within $\pm\tau$ samples. This is alert-level recall per signature: $\text{alert_cov}_i = \text{tp_alerts}_i / n_{\text{alerts},i}$. The reported value is the mean of this per-signature recall across all alerted signatures.

7.2 Results

Table 18 reports aggregate replay metrics.

Storage reduction. All four statistical detectors reduce trace volume by ≈ 89–91%—consistent with the main evaluation's storage reduction (Table 3). EWMA

Method	Stor. Red. (%)	Trigger Freq. (%)	Alert Coverage (%)	Burst Ratio	Activations
ARIMA	89.75 (7.31)	5.82 (1.78)	85.66 (34.91)	0.515	10.28
LAST	89.42 (3.03)	5.65 (1.49)	70.22 (45.70)	0.633	9.96
SMA	88.83 (6.03)	5.76 (1.83)	**93.01 (25.22)**	0.574	10.18
EWMA	**91.00 (2.71)**	6.07 (1.74)	79.04 (40.62)	0.453	10.72

Table 18: Trace-on replay aggregate metrics across 214 signatures for the four statistical detectors. 136 of 214 signatures have ≥ 1 alert in the test set. Best storage reduction in bold; best coverage in bold.

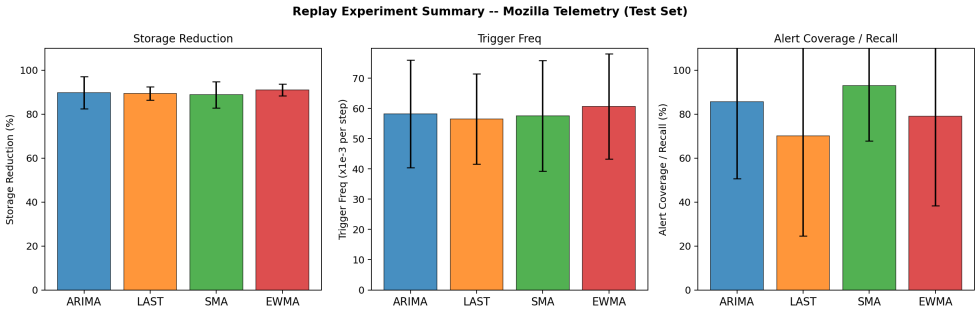


Fig. 9: Boxplot distributions of storage reduction, trigger frequency, and alert coverage across 214 signatures for the four statistical detectors.

achieves marginally higher reduction (91.0%) because its exponential weighting reacts more slowly to isolated spikes, resulting in less time in the ON state. LAST has low standard deviation (3.0%) indicating highly consistent storage savings across signatures.

Alert coverage (mean per-signature alert recall). SMA achieves the highest mean alert recall (93.0%), meaning that across alerted signatures SMA's trace-ON windows cover on average 93.0% of ground-truth alerts in the test period. ARIMA follows at 85.7%. EWMA's conservative activation leads to the lowest mean alert recall among structured methods (79.0%); LAST is worst (70.2%) despite similar trigger frequency. The gap between SMA (93.0%) and EWMA (79.0%) directly reflects the detection-rate difference seen in Table 3.

Trigger frequency and burst ratio. The four statistical methods activate 5.7–6.1% of total test samples. The *burst ratio* (mean interval duration relative to activation count) is lowest for EWMA (0.453), ARIMA and SMA display intermediate behaviour (0.515–0.574), and LAST shows the longest average activation intervals (0.633).

Figure 9 shows boxplot distributions of storage reduction, trigger frequency, and alert coverage across all signatures.

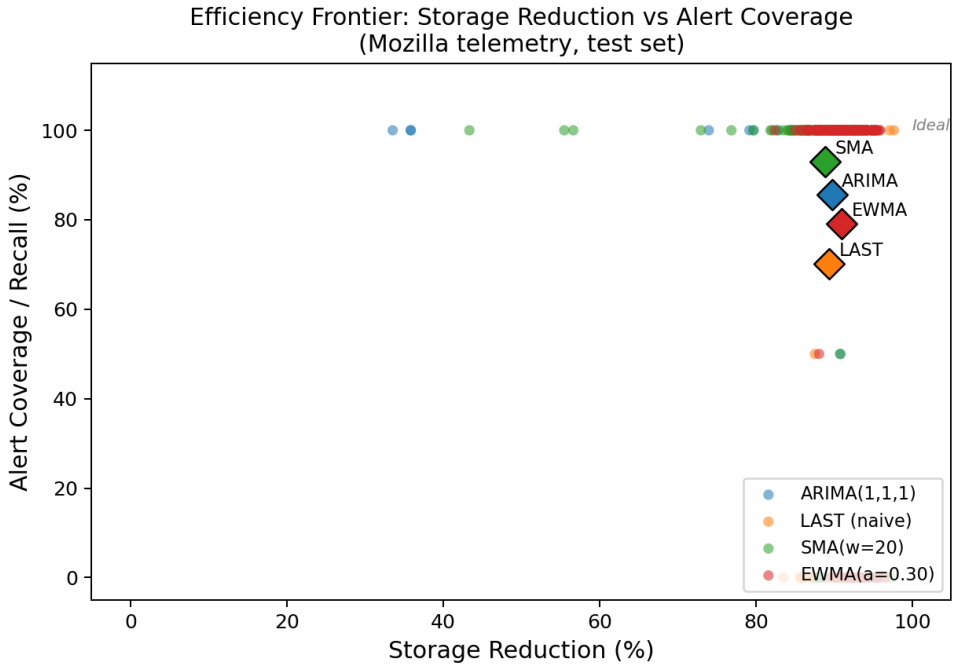


Fig. 10: Efficiency frontier: storage reduction vs. alert coverage. A Pareto-optimal detector lies at the top-right corner. SMA achieves the best combined storage–coverage trade-off; EWMA maximises storage reduction at the cost of lower alert coverage.

7.3 Efficiency Frontier Analysis

Figure 10 plots storage reduction against alert coverage for each of the four statistical detectors, revealing the storage–coverage Pareto frontier.

SMA dominates for coverage; EWMA dominates for storage reduction. ARIMA represents a balanced choice: second-highest coverage (85.7%) with storage reduction comparable to SMA (89.75% vs. 88.83%). LAST is dominated—it achieves neither the highest storage reduction nor the highest coverage.

The CDF of per-signature storage reduction is shown in Figure 11: all four statistical methods show tight distributions of storage reduction (85–95%).

RQ4 summary: All four statistical detectors achieve a storage reduction of ≈ 89–91%. The critical differentiator is *alert coverage*: SMA achieves the highest mean per-signature alert recall (93.0%)—it covers on average 93.0% of a signature's test-set alerts—while LAST achieves only 70.2%. SMA dominates the coverage dimension (93.0% mean alert recall at 88.83% storage reduction) and is the recommended choice for deployments where alert coverage is the primary objective. ARIMA provides the second-highest coverage (85.7%) at comparable storage reduction (89.75%), and may be preferred when an interpretable forecasting model is required; but it should not be selected over SMA on the basis of performance data from this benchmark. LAST

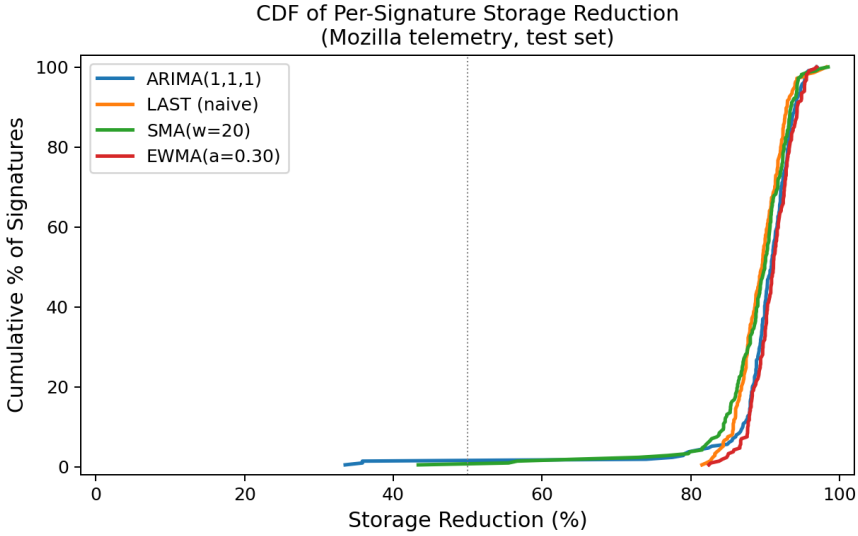


Fig. 11: Cumulative distribution of per-signature storage reduction. All methods concentrate at 85–95%.

is Pareto-dominated, achieving neither the highest storage reduction nor the highest coverage.

8 Discussion

8.1 Operational Engine Selection

The experimental results support the following guidance for practitioners deploying the DAET framework. Each recommendation below cites numbers from their source protocol explicitly: the canonical 60%/20%/20% protocol (Section 5.6) supplies the primary F1 and storage-reduction comparisons; the calibration baseline (Section 5.4) supplies recall, detection-rate, and statistical-significance results; and the replay experiment (Section 7) supplies alert-coverage estimates, all on the calibration-baseline split. Cross-protocol conclusions are stated separately in Section **??**. Three deployment profiles emerge depending on accuracy objectives and resource constraints.

Recommended default (most deployments): use SMA. *Canonical 60%/20%/20% protocol*: SMA achieves F1 = 0.104 on Firefox-Android ($w = 40$) and F1 = 0.085 on mozilla-beta ($w = 20$)—the leading or co-leading F1 on both datasets (Tables 7–8). *Calibration baseline (70%/30%, 214 signatures)*: SMA ($w = 20$) achieves detection rate 96%, mean recall 0.657, and F1 = 0.186 (Table 3). *Replay experiment (calibration-baseline split)*: mean per-signature alert coverage 93.0% and storage reduction 88.83% (Table 18). It requires no model fitting, is deterministic, runs in 30 seconds on all 214 signatures, and uses 16× less peak memory than ARIMA. The optimal window (w) is dataset-dependent: $w = 40$ gave the best validation-set F1 on

Firefox-Android under the canonical protocol, $w = 20$ on mozilla-beta canonical and on the calibration baseline; deployments should validate w on a held-out segment.

If explicit trend modelling or extensibility is required: use ARIMA(1,1,1). It significantly outperforms LAST and EWMA, matches SMA in F1 and precision, and provides a statistically motivated prediction model that distinguishes gradual workload evolution from sudden performance regressions. ARIMA(1,1,1) can be extended to ARIMA(p,d,q) by adaptive order selection (AIC/BIC), allowing tuning to the autocorrelation structure of specific signatures. Teams should be aware of the 27× runtime cost relative to SMA (Table 4) when making this choice.

If storage reduction is the primary constraint: use EWMA on Firefox-Android. With tuned $\alpha = 0.05$ (Section 5.6), EWMA achieves higher storage reduction than SMA on Firefox-Android (89.4% vs. 88.3%), while matching SMA in F1 (0.103 vs. 0.104) and maintaining comparable detection quality. On mozilla-beta, the margin reverses: SMA achieves 88.8% storage reduction vs. EWMA's 88.5%, so the storage advantage is dataset-dependent and shouldn't be taken as a universal rule. EWMA with $\alpha = 0.05$ is a practical alternative to SMA when higher storage efficiency is a priority and the deployment context matches Firefox-Android characteristics.

LAST should generally be avoided: it is significantly worse than all other methods in precision, recall, and F1, and achieves the lowest alert coverage (70.2%) in the replay experiment.

If a learned Transformer predictor is operationally preferred: use the compact Transformer under the 60%/20%/20% protocol with dataset-specific HP tuning (Section 5.6). On Firefox-Android (WINDOW=20, $d_{\text{model}} = 16$), five seeds yield F1 = 0.105 (±0.002) and detection rate 74.4%—competitive with tuned SMA. On mozilla-beta (WINDOW=10, $d_{\text{model}} = 32$), F1 = 0.079 (±0.001) and detection rate 89.8%. Storage reduction is lower (67–71% vs. 88–90% for classical methods) because the learned predictor activates trace-on windows more frequently at the standard anomaly-score threshold; practitioners should calibrate the threshold for storage-constrained deployments.

8.2 ARIMA vs. SMA: Why Not Simply Use SMA?

A natural question arises: if SMA and ARIMA are not significantly different in F1 and precision on the calibration baseline (Wilcoxon $p > 0.5$; Section 5.5), and SMA leads on recall and detection rate (also calibration baseline), why retain ARIMA as a reference engine?

We identify three engineering-motivated reasons: (1) SMA's moving average generates a prediction only after the window is fully populated and reacts symmetrically to both up and down deviations. ARIMA(1,1,1)'s integrated first-order model can capture monotonic trends (e.g., gradual CPU drift) and will continue predicting along the new trend after a level shift, thereby reducing spurious alarms for organic load growth. (2) In settings where non-stationarity is pronounced (servers with daily seasonality, progressive memory leaks), the integrated component of ARIMA provides a systematic de-trending mechanism that SMA lacks. (3) ARIMA(1,1,1) can be extended to ARIMA(p,d,q) by adaptive order selection (AIC/BIC), allowing the system to be tuned to the autocorrelation structure of each specific signature.

Importantly, these reasons are conceptual and engineering-oriented, not empirical: the current benchmark does not provide evidence that ARIMA's trend-modelling advantage manifests in measurably better detection on Mozilla telemetry. Readers who prioritise measured accuracy over interpretability or extensibility should consider SMA as the deployment default.

8.3 When SMA Is the Better Choice

SMA retains the highest recall (0.657), detection rate (96%), and alert coverage in the replay experiment (93%) among all methods on the calibration baseline (Section 5.4). Under the canonical 60%/20%/20% protocol (Section 5.6), SMA ($w = 40$) achieves F1 = 0.104 on Firefox-Android, and EWMA ($\alpha = 0.05$) matches it at F1 = 0.103 with marginally higher storage reduction. Deployment teams requiring maximum recall and alert coverage should prefer SMA, with the window validated on a held-out segment:

- Highest recall (0.657) and detection rate (96%) among all methods *(calibration baseline, Section 5.4)*.
- Highest replay alert coverage (93.0%) *(replay experiment, calibration-baseline split, Table 18)*.
- F1 = 0.104 (Firefox-Android, $w = 40$) and 0.085 (mozilla-beta, $w = 20$) *(canonical 60%/20%/20% protocol, Tables 7–8)*.
- Zero training time (deterministic, $O(1)$ memory).

Across both protocols, SMA is consistently the highest-recall and highest-coverage method; the optimal window differs by dataset and protocol and should be set by validation. For deployments where storage reduction is also a priority, EWMA($\alpha = 0.05$) achieves comparable canonical F1 to SMA (Firefox-Android: 0.103 vs. 0.104; mozilla-beta: 0.083 vs. 0.085) with 89.4%–88.5% storage reduction vs. SMA's 88.3%–88.8%; however, EWMA's replay alert coverage (79.0%) is lower than SMA's (93.0%), so the trade-off is explicit.

8.4 Precision Ceiling

Precision is low (< 0.12) for all methods. This is inherent to the dataset: Treeherder regression alerts are based on a different detection algorithm (Bayesian change-point detection), and many short performance excursions that DAET detects are *real* performance anomalies that simply did not persist long enough for Mozilla's system to raise a formal regression ticket. The metric captures *how many of our detection windows coincide with official alerts*, not whether the detected anomalies are genuine. We therefore distinguish two precision concepts: "Treeherder agreement precision" (what Tables 3–8 report) and "true anomaly precision" (the fraction of detections corresponding to genuine performance anomalies). The true anomaly precision is unknown without manual inspection. A stronger validation would require (i) manual inspection of a random sample of alleged false positives to classify them as genuine anomalies vs. noise, or (ii) cross-referencing with a second alert source (e.g., Bugzilla regression tickets) to identify detections that correspond to real events not yet labelled by Treeherder. We acknowledge this as a limitation and note it as directions for future work.

8.5 Scientific Conclusions vs. Deployment Recommendations

It is important to distinguish what the evidence supports scientifically from what it supports as a deployment recommendation.

What the evidence shows: All metrics throughout this paper measure *Treeherder-agreement*: how well a detector's binary output aligns with Mozilla's automated Bayesian alert system. This is not the same as measuring true anomaly detection accuracy or debugging effectiveness. Subject to that construct constraint: the canonical evaluation (Section 5.6) shows that SMA and properly tuned EWMA lead on Treeherder-agreement F1 on two repositories under a fair 60%/20%/20% protocol; ARIMA is third; LAST is weakest. The calibration baseline (Section 5.4) confirms the ranking on a 70/30 split: SMA(w=20) reaches recall 0.657, F1 0.186, and detection rate 96.0%; ARIMA is statistically indistinguishable from SMA in F1 ($p = 0.547$) and precision ($p = 0.652$). The compact Transformer under the canonical protocol achieves F1 = 0.105 ($\pm$0.002) on Firefox-Android and F1 = 0.079 ($\pm$0.001) on mozilla-beta, showing that a learned backbone is competitive with classical methods on this alert-agreement benchmark.

What the evidence does not show: The evidence does not show that any single method is best for all workloads or all deployment contexts. The benchmark is limited to one application (Firefox-Android), one telemetry platform (Treeherder), and one operating period. Generalisation to other systems, seasonality regimes, or alert-generation protocols requires further evaluation.

Deployment recommendation: For deployments where recall and alert coverage are the primary objectives, SMA is the recommended default. Under the canonical 60%/20%/20% protocol, SMA($w = 40$) achieves F1 = 0.104 and detection rate 72.8% on Firefox-Android. Under the calibration baseline (for deployments matching the conference-paper setup), SMA($w = 20$) achieves recall 0.657, F1 0.186, and detection rate 96.0% (30 s runtime, 16$\times$ less memory than ARIMA). For a learned predictor, the compact Transformer under the canonical protocol is the recommended choice (F1 = 0.105 on Firefox-Android, 0.079 on mozilla-beta). ARIMA(1,1,1) is appropriate when explicit trend modelling, interpretable residuals, or extensibility to higher-order variants are required; its 27$\times$ runtime overhead (Table 4) should be factored into that decision.

9 Threats to Validity

Internal validity. The four case studies (Section 4) are illustrative demonstrations using known bug reports. While the framework's candidate list was generated independently of the bug report, the evaluation is retrospective and may not generalise to bugs of a different character. The canonical comparative evaluation (Section 5.6) uses a strict 60%/20%/20% chronological split on two repositories with an independent held-out validation set for SMA, EWMA, and Transformer hyperparameter selection; the test set is evaluated exactly once. Two further internal validity limitations remain. First, the shared anomaly-score parameters ($\beta = 0.30$, $\theta = 0.30$) are applied identically to all methods without per-method or per-dataset tuning; the ablation study shows both parameters produce large swings in precision/recall trade-offs, so

the current ranking may shift under individual-method optimisation of these shared parameters. Second, the EWMA validation curve is monotonically decreasing on both datasets, meaning the selected $\alpha = 0.05$ sits at the lower boundary of the tested grid; the true optimal α may be smaller, and the reported EWMA results are best-within-grid rather than globally optimal. The calibration baseline (Section 5.4) replicates the conference paper's 70%/30% setup but without an independent validation hold-out, making its SMA/EWMA parameter choices an acknowledged internal validity limitation; it is retained solely for prior-work comparison. The bootstrap confidence intervals and Wilcoxon tests (Section 5.5) are computed on the calibration-baseline per-signature scores for prior-work comparison. Bootstrap confidence intervals and Wilcoxon signed-rank tests on the canonical 60%/20%/20% test data are reported in Section 5.7 (Tables 9–12). The compact Transformer hyperparameters were tuned on the held-out validation set; the test set was evaluated once. Results are reported in Section 5.6 and Tables 7–8: on Firefox-Android the compact Transformer achieves F1 = 0.105 ($\pm$0.002) and detection rate 74.4%; on mozilla-beta F1 = 0.079 ($\pm$0.001), slightly below tuned SMA and EWMA.

Construct validity. A critical limitation is that this paper evaluates *agreement with Treeherder alerts*, not improvement in actual performance-debugging effectiveness. Precision, recall, and F1 are computed relative to Treeherder's Bayesian change-point alerts, which are themselves imperfect: they may miss transient regressions, include false positives from infrastructure changes, and are generated by a different detection algorithm with different sensitivity settings. Consequently, the paper does not show that DAET improves engineer productivity or reduces time-to-root-cause—it shows that DAET's binary output is correlated with an existing automated alerting system. Whether higher Treeherder-agreement precision corresponds to genuine improvement in debugging utility would require a controlled user study or a longitudinal analysis linking DAET-triggered traces to bug-fix time. The tolerance window $\tau = 5$ samples introduces further matching leniency. All four statistical detectors use the same evaluation protocol, so the *comparative* findings are unaffected by this construct choice, but all absolute precision and recall values should be interpreted as "Treeherder-agreement metrics" rather than true anomaly precision (see the Precision Ceiling discussion in Section 8). The false-positive counts have not been manually adjudicated: a detection event classified as a false positive by Treeherder alignment may correspond to a genuine but unlabelled performance excursion. A stricter validation would require manual inspection of a random sample of alleged false positives, or cross-referencing with a second alert source (e.g., Bugzilla regression tickets).

External validity. The evaluation uses two Mozilla Treeherder repositories (Firefox-Android and mozilla-beta); both are from the same organisation and CI platform, which limits generalisability to other applications, tracing granularities, or monitoring infrastructures. Furthermore, because all metrics measure agreement with Treeherder alerts rather than ground-truth anomaly validity, the consistent qualitative ranking across both repositories should be treated as *preliminary*, not conclusive, evidence of cross-dataset robustness: it is possible that both benchmarks share the same systematic bias relative to true anomaly ground truth. Validation on external CI

systems using an independent ground truth (e.g., manually adjudicated or Bugzilla-confirmed regressions) is required before stronger generalisability claims can be made. The replay experiment simulates storage savings analytically; actual deployment savings depend on storage backend, compression, and I/O scheduling.

Conclusion validity. Statistical tests are reported for both the calibration baseline (Section 5.5, Wilcoxon on 214 paired observations) and the canonical 60%/20%/20% test set (Section 5.7, Wilcoxon on 342 Firefox-Android and 1,477 mozilla-beta observations). The Bonferroni-corrected threshold for six pairwise comparisons is $\alpha/6 = 0.0083$. All conclusions stated as significant in Tables 11–12 meet this stricter bound. The larger n for the canonical tests (342 and 1,477) provides substantially more power than the calibration-baseline $n = 214$; the key qualitative finding that SMA $\approx$ EWMA and both outperform LAST is confirmed at $p < 0.001$ on both datasets.

10 Conclusion

This paper extended the Dynamic Adaptive Execution Tracing (DAET) framework originally introduced in [6] with five new contributions: a large-scale comparative evaluation of four classical change-detection engines, a compact Transformer encoder under fair multi-seed protocol, a formal statistical analysis, a trace-on replay experiment, and a three-way chronological split experiment. The journal contribution is positioned as an *empirical comparative extension*: the core DAET framework architecture was established in the conference paper; this work provides the large-scale prospective validation and comparative detector study that was explicitly deferred as future work.

The framework's three-phase architecture—function selection via family-group analysis and CoV ranking; change detection via a pluggable change-detection engine with a dynamic anomaly-score threshold; and trace-configuration gating—was validated qualitatively through four Firefox Bugzilla case studies and quantitatively via a unified 60%/20%/20% pipeline on 342 Firefox-Android and 1,477 mozilla-beta Treeherder telemetry signatures across five method families; a calibration baseline on 214 signatures under the conference paper's 70%/30% setup is also reported for prior-work comparison.

The canonical evaluation (Section 5.6, `run_full_evaluation.py`) showed that on Firefox-Android (60%/20%/20%, independent validation tuning) SMA($w = 40$) achieves F1 = 0.104 and detection rate 72.8%; EWMA($\alpha = 0.05$) matches it at F1 = 0.103 with marginally higher storage reduction (89.4%). On mozilla-beta (1,477 signatures), SMA($w = 20$) leads at F1 = 0.085 and detection rate 91.7%; EWMA achieves F1 = 0.083. The qualitative ranking (SMA $\approx$ EWMA $\geq$ ARIMA $>$ LAST) is consistent across both datasets on the Treeherder-agreement benchmark, providing preliminary evidence of robustness within the Mozilla CI context; stronger generalisability claims require validation against an independent ground truth. The compact Transformer (five seeds, canonical protocol, dataset-specific HP tuning) achieves F1 = 0.105 ($\pm$0.002) and detection rate 74.4% on Firefox-Android—competitive with tuned SMA—and F1 = 0.079 ($\pm$0.001) on mozilla-beta. The calibration baseline

(Section 5.4, 70%/30%, 214 signatures) found SMA(w=20) achieving recall 0.657 and F1 0.186; statistical tests confirm ARIMA is indistinguishable from SMA in F1 ($p = 0.547$) and precision ($p = 0.652$), and significantly outperforms LAST and EWMA in precision, recall, and F1 (all $p \leq 0.002$; ARIMA–LAST FAR difference not significant, $p = 0.765$). ARIMA(1,1,1) is an appropriate alternative when explicit trend modelling, interpretable residuals, or extensibility to higher-order ARIMA variants are operationally required; at 27× the runtime cost and 16× the memory footprint of SMA (Table 4), it should not be used as the default engine on the basis of measured accuracy alone. The replay experiment (Section 7) confirmed that all four statistical detectors reduce trace storage by 89–91% on the calibration-baseline test split, with SMA achieving the highest mean per-signature alert recall (93.0%) and EWMA the highest storage reduction (91.0%). The ablation study (Section 6) confirmed that the default parameters ($\beta = 0.30$, $\theta = 0.30$, ARIMA(1,1,1), $\tau = 5$) are an operationally justified high-recall operating point, not globally optimal settings; under the ablation's stricter regression-only GT, $\beta = 0.20$ yields ARIMA F1 = 0.266 vs. the default 0.144 (vs. 0.174 under the main-table GT), and $\theta \geq 0.40$ yields F1 = 0.27 at the cost of lower recall.

Future work. Open directions include: (i) re-implementing the replay experiment within the unified pipeline (`run_full_evaluation.py`) to fully consolidate the deployment evidence under the canonical 60%/20%/20% protocol; (ii) validating the low-precision findings against a manually annotated subset of alleged false positives and against Bugzilla regression reports to estimate true anomaly precision; (iii) extending the framework to seasonally non-stationary workloads using SARIMA or STL decomposition; (iv) integrating anomaly score signals directly into an observability platform (e.g., Prometheus/Grafana) for real-time deployment; (v) evaluating the framework on additional CI/CD pipelines beyond Mozilla Firefox; and (vi) extending the dual-dataset evaluation to additional Mozilla CI projects and to external production systems beyond Firefox to assess cross-domain generalisability of the qualitative ranking.

A Reproducibility Details for Qualitative Case Studies

This appendix records the workload and measurement specifics for the four qualitative case studies (Section 4) to support independent replication.

Case Study 1 — Apache/nmon Setup

- **Workload:** PHP script served by Apache; 30% probability of loading a 50 MB plain-text file and echoing it back, plus random prime-number generation for CPU load. Concurrent request count was stepped up fivefold (starting at 2, incremented by 2) over the course of the run.
- **Data collection command:** `nmon -f -s 1 -c 3600` (1-second sampling interval, 3,600 samples = 1 hour; output written to the current directory rather than the terminal via the `-f` flag).
- **Duration:** 1 hour.

- **ARIMA parameters:** $\text{threshold}_{\text{ARIMA}} = 3\%$ (both upward and downward); $\beta = 0.30$; $\text{threshold}_{\text{anomaly}} = 0.30$.

Case Studies 2 and 3 — Firefox Application Profiling Setup

- **Firefox build:** built from source with the `perf` profiling flag enabled (following Mozilla's official documentation [34]) to allow JIT-compiled code to be captured in call stacks.
- **Profiling tool:** Linux `perf` with 10 ms time-slice snapshots to attribute execution time to individual function calls.
- **Profiling duration:** 20 minutes (Case Study 2 — large/small PDF combinations); 15 minutes (Case Study 3 — 3D CSS animation sequences).
- **Cutoff:** 5% of all unique family-groups retained for monitoring (sorted by call frequency then CoV ranking).

Case Study 4 — Firefox DNS/Network Profiling Setup

- **Workload:** Firefox launched repeatedly while the system DNS was rotated among Google's DNS, a random invalid DNS, and disconnected network to reproduce the DNS-resolution failure described in [31].
- **Profiling tool:** Linux `perf` (same configuration as Case Studies 2 and 3).
- **Profiling duration:** 15 minutes.
- **Cutoff:** 5% CoV-ranking cutoff; 289 monitored items from 5,764 unique family-groups.

Acknowledgements. The authors thank the Mozilla Performance Team for making the Treeherder telemetry dataset publicly available.

Declarations

Conflict of interest: The authors declare no conflict of interest.
Data and code availability: All source code for the experimental pipeline, evaluation, ablation, replay experiments, and figure generation is publicly available at https://github.com/ghazalkhb/Performance-Debugging. A tagged release corresponding to the exact version used for this paper is provided. The repository includes:

- `requirements.txt` with pinned package versions (Python 3.12, statsmodels 0.13.5, numpy 1.24, pandas 2.0, scipy 1.11, torch 2.4.1+cpu);
- `run_full_evaluation.py`: unified dual-dataset evaluation pipeline that runs all five methods (ARIMA, LAST, SMA, EWMA, and compact Transformer) on both datasets under the 60%/20%/20% protocol and reproduces all three-way tables and figures with a single command;
- preprocessing instructions for obtaining and filtering the Mozilla Treeherder dataset (signature matching, length filters, alert alignment);
- all random seeds are fixed to 42 in the bootstrap resampling; the detectors themselves are deterministic.

The Mozilla performance-telemetry datasets are publicly accessible via the Treeherder API at https://treeherder.mozilla.org/ under the projects `firefox-android` and `mozilla-beta`.

References

[1] Cantrill, B., Shapiro, M. W. & Leventhal, A. H. *Dynamic Instrumentation of Production Systems*, 15–28 (2004).

[2] Desnoyers, M. & Dagenais, M. R. *The lttng tracer: A low impact performance and behavior monitor for gnu/linux*, Vol. 2006, 209–224 (Citeseer, 2006).

[3] Rostedt, S. *Ftrace linux kernel tracing* (2010).

[4] Gebai, M. & Dagenais, M. R. Survey and Analysis of Kernel and Userspace Tracers on Linux. *ACM Computing Surveys* **51**, 1–33 (2018). URL http://dx.doi.org/10.1145/3158644.

[5] Arulraj, J., Chang, P.-C., Jin, G. & Lu, S. Production-run software failure diagnosis via hardware performance counters. *Acm Sigplan Notices* **48**, 101–112 (2013).

[6] Khan, M. A. & Ezzati-Jivan, N. *Multi-level Adaptive Execution Tracing for Efficient Performance Analysis*, 104–109 (2023).

[7] Calcagno, C. *et al. Moving Fast with Software Verification*, 3–11 (Springer, 2015).

[8] Hovemeyer, D. & Pugh, W. Finding Bugs is Easy. *ACM SIGPLAN Notices* **39**, 92–106 (2004).

[9] Jin, G., Song, L., Shi, X., Scherpelz, J. & Lu, S. Understanding and detecting real-world performance bugs. *ACM SIGPLAN Notices* **47**, 77–88 (2012).

[10] Nistor, A., Chang, P.-C., Radoi, C. & Lu, S. *Caramel: Detecting and Fixing Performance Problems That Have Non-intrusive Fixes*, Vol. 1, 902–912 (IEEE, 2015).

[11] Dai, T., Dean, D., Wang, P., Gu, X. & Lu, S. Hytrace: A hybrid approach to performance bug diagnosis in production cloud infrastructures. *IEEE Transactions on Parallel and Distributed Systems* **30**, 107–118 (2018).

[12] Mace, J., Roelke, R. & Fonseca, R. *Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems*, 378–393 (2015).

[13] Mirgorodskiy, A. V. *Automated problem diagnosis in distributed systems*. Ph.D. thesis, University of Wisconsin–Madison (2006).

[14] Dean, D. J. *et al.* *Perfscope: Practical Online Server Performance Bug Inference in Production Cloud Computing Infrastructures*, 1–13 (2014).

[15] Nguyen, H., Shen, Z., Tan, Y. & Gu, X. *FChain: Toward Black-box Online Fault Localization for Cloud Systems*, 21–30 (2013).

[16] Zhang, S., Liu, D., Zhou, L., Ren, Z. & Wang, Z. *Diagnostic Framework for Distributed Application Performance Anomaly Based on Adaptive Instrumentation*, 164–169 (IEEE, 2020).

[17] Tate, A. B. & Bewoor, L. A. Optimization of system's performance with kernel tracing by cohort intelligence. *International Journal of Information Technology and Computer Science(IJITCS)* (2017).

[18] Ehlers, J. & Hasselbring, W. Self-Adaptive Performance Monitoring of Service-Oriented Systems. *SOCA* **5**, 225–242 (2011).

[19] Ates, E. *et al.* *An Automated, Cross-layer Instrumentation Framework for Diagnosing Performance Problems in Distributed Applications*, 165–170 (2019).

[20] Liu, D. *et al.* *Opprentice: Towards Practical and Automatic Anomaly Detection Through Machine Learning*, 211–224 (ACM, 2015).

[21] Lavin, A. & Ahmad, S. *Evaluating Real-time Anomaly Detection Algorithms – The Numenta Anomaly Benchmark*, 38–44 (IEEE, 2015).

[22] Laptev, N., Amizadeh, S. & Flint, I. *Generic and Scalable Framework for Automated Time-series Anomaly Detection*, 1939–1947 (ACM, 2015).

[23] Cid-Fuentes, J. A., Szabo, C. & Falkner, K. Adaptive performance anomaly detection in distributed systems using online svms. *IEEE Transactions on Dependable and Secure Computing* **17**, 928–941 (2018).

[24] Laguna, I. *et al.* Automatic problem localization via multi-dimensional metric profiling (2013).

[25] Munawar, M. A., Jiang, M., Reidemeister, T. & Ward, P. A. Filtering system metrics for minimal correlation-based self-monitoring (2009).

[26] Dymshits, M., Myara, B. & Tolpin, D. Process monitoring on sequences of system call count vectors (2017).

[27] Han, S., Dang, Y., Ge, S., Zhang, D. & Xie, T. *Performance Debugging in the Large via Mining Millions of Stack Traces*, 145–155 (IEEE, 2012).

[28] Valsorda, F. Linux System Call Table. https://filippo.io/linux-syscall-table/ (2014).

[29] clessili. Bug 1768481 (2022). URL https://bugzilla.mozilla.org/show_bug.cgi?id=1768481.

[30] Muizelaar, J. Bug 1637586 (2020). URL https://bugzilla.mozilla.org/show_bug.cgi?id=1637586.

[31] Cook, J. Bug 1776469 (2022). URL https://bugzilla.mozilla.org/show_bug.cgi?id=1776469.

[32] Mozilla Foundation. Treeherder: Mozilla Performance Testing and Alerting Platform. https://treeherder.mozilla.org/ (2023). Accessed: 2024.

[33] Seabold, S. & Perktold, J. Statsmodels: Econometric and statistical modeling with Python. *9th Python in Science Conference* (2010).

[34] Mozilla. Jit profiling with perf — firefox source docs documentation. URL https://firefox-source-docs.mozilla.org/performance/jit_profiling_with_perf.html.